ModusToolbox™ Software Training: Level 1 - Getting Started

Chapter 2: Tools

After completing this chapter, you will understand each of the ModusToolbox™ tools, what they do, and how they fit together to give you a complete development flow.

Convention	Usage	Example
`Courier New`	Displays code and text commands	`CY_ISR_PROTO(MyISR);` `make build`
Italics	Displays file names and paths	*sourcefile.hex*
[**bracketed, bold**]	Displays keyboard commands in procedures	[**Enter**] or [**Ctrl**] [**C**]
Menu > Selection	Represents menu paths	**File > New Project > Clone**
Bold	Displays GUI commands, menu paths and selections, and icon names in procedures	Click the **Debugger** icon, and then click **Next**.

Table of contents

Table of contents **2**
1 Software, application, and BSP versions **4**
2 Tools and the make system **4**
2.1 Make 5
2.2 Using the command line interface (CLI) 5
2.2.1 Windows 6
2.2.2 macOS / Linux 6
2.2.3 Integrated Terminal Panel 7
2.2.4 Make targets (commands) 7
2.2.5 Make variables 8
2.2.6 Accessing tools from the command line 9
3 Dashboard **10**
4 Windows Subsystem for Linux (WSL) **11**
5 Application creation **11**
5.1 Project Creator tool GUI 11
5.2 Project Creator CLI 15
5.3 Template processing 15
6 Serial Terminal Emulator **16**
6.1 Determine the Port 16
6.2 PuTTY (Windows and Linux) 17
6.2.1 Screen (MacOS and Linux) 18
7 Exercises (part 1) **19**
Exercise 1: Create, build, program and debug a Hello World application in VS Code 19
Exercise 2: Build and program using the Command Line Interface 23
8 Board Support Packages (BSPs) **24**
8.1.1 BSP directory structure 24
8.2 BSP dependency version locking 26
8.3 BSP names 26
8.4 BSP documentation 26
9 Application directory organization **27**
9.1 Single-core application 27
9.1.1 Makefile 28
9.2 Multi-core applications 30
9.2.1 Makefile and make targets 31
10 Library management **33**
10.1 Library classification 33
10.1.1 Shared vs. local 33
10.1.2 Direct vs. indirect 33
10.1.3 Fixed vs. dynamic versions 34
10.2 Including libraries and dependencies 34
10.2.1 Including libraries that are specified in a manifest 34
10.2.2 Including libraries are not specified in a manifest 35
10.2.3 Specifying dependencies for custom BSPs 36
10.3 Library Manager tool 36
10.3.1 Active BSP 39
10.4 Re-Downloading Libraries 39

Table of contents

11 Configurators 40
11.1 BSP configurators 40
11.1.1 Device Configurator 41
11.1.2 CAPSENSE™ configurator and tuner 43
11.1.3 Smart I/O configurator 44
11.1.4 QSPI configurator 45
11.1.5 SegLCD configurator 45
11.2 Library configurators 46
11.2.1 Bluetooth® configurator 46
11.2.2 USB configurator 46
12 Working with custom BSPs 47
12.1 BSP Assistant 47
12.1.1 Opening the tool 47
12.1.2 Open an existing BSP 48
12.1.3 Create a new BSP from an existing one 48
12.1.4 Create a new BSP based on the device part numbers 49
12.1.5 GUI description 50
12.1.6 Save, Undo and Redo 54
12.1.7 Updating launch settings 54
12.2 Using a custom BSP in a new application 54
12.2.1 Using a local BSP 54
12.2.2 Using a custom manifest for the BSP 55
12.2.3 Using a version-controlled copy of the BSP 55
13 Creating applications for other IDEs 56
14 Version control and sharing applications 56
14.1 Files to include/exclude 56
14.2 Using version control software 56
14.3 Manual file copy 57
14.4 Using the Archive tool 57
15 Manifests 59
16 Troubleshooting 60
16.1 Build support not found 60
16.1.1 Path too long 60
16.2 Incorrect device 60
17 Network considerations 61
17.1 Network proxy settings 61
17.2 Offloading manifest files 62
17.3 Local content 62
18 Compiler optimization 63
19 Exercises (part 2) 64
Exercise 3: Use the Device Configurator to change BSP settings 64
Exercise 4: Use the BSP Assistant 65
Disclaimer 66

1 Software, application, and BSP versions

This training focuses on ModusToolbox™ 3.x tools. Applications and BSPs created with the ModusToolbox™ 2.x ecosystem use a slightly different format. They will fully function using the 3.x tools, but you will be unable to take advantage of all of the 3.x features.

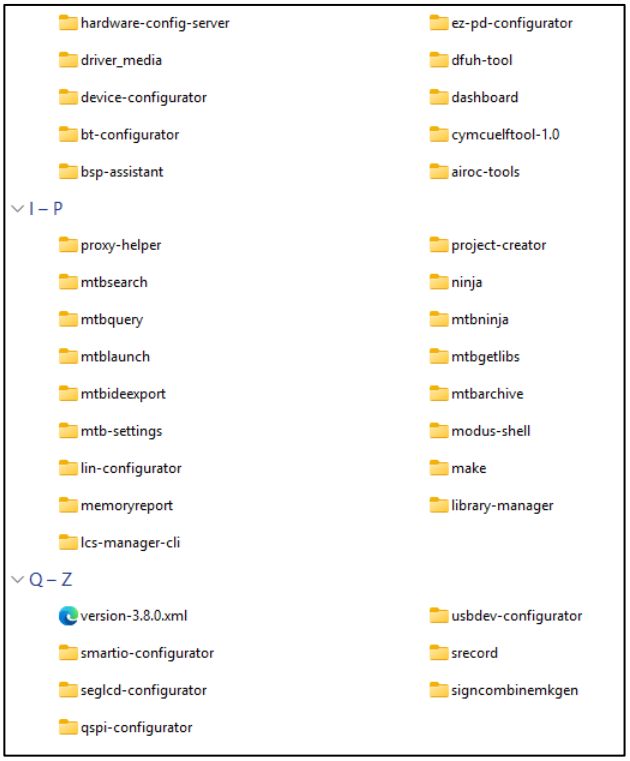
2 Tools and the make system

As you learned in the last chapter, the ModusToolbox™ ecosystem consists of a set of tools as well as software assets. In this chapter, we will go through the most used tools and examine each type of asset.

The list of tools may change over time as new tools are added. However, tools that are installed by the ModusToolbox™ tools installer are located by default in the following location:

~/ModusToolbox/tools_<version>

The directory will look like this:



Each tool has its own subdirectory containing the tool itself and documentation. In many cases, tools will have both a GUI version and a command line version.

You can launch many tools from inside an IDE (this will be explained in more detail later). In Windows, there are **Start** menu options for each tool or you can enter the tool name in the Windows search box. On all systems, you can run tools directly from the tool's installation directory or from a command line interface.

2.1 Make

One of the directories under tools is *make*. Many operations in the ModusToolbox™ ecosystem are built upon the GNU Make system (https://www.gnu.org/software/make/), so it is a fundamental part of the ecosystem. In fact, even when you use the Visual Studio Code IDE or Eclipse for ModusToolbox™, many operations just call the underlying `make` operation so that you get the same behavior no matter how you run the operation. For example, to build from the command line, you use the command `make build`. When you build the same application from inside VS Code or Eclipse, it just calls the same `make build` command for you. This is important because it means you can move back and forth between IDE operations and command line operations with the security of knowing that the results won't change.

The `make` process gets its instructions from a makefile. It contains variables and settings that control what `make` does. There is a top-level makefile in each ModusToolbox™ application with the name *Makefile*. That's the first file that `make` looks at. This top-level file can include other makefiles, which typically have the file extension .mk. For example, an application's Makefile will include the *start.mk* file from the tools directory. Each BSP also has a makefile, called *bsp.mk*, that gets included during the build.

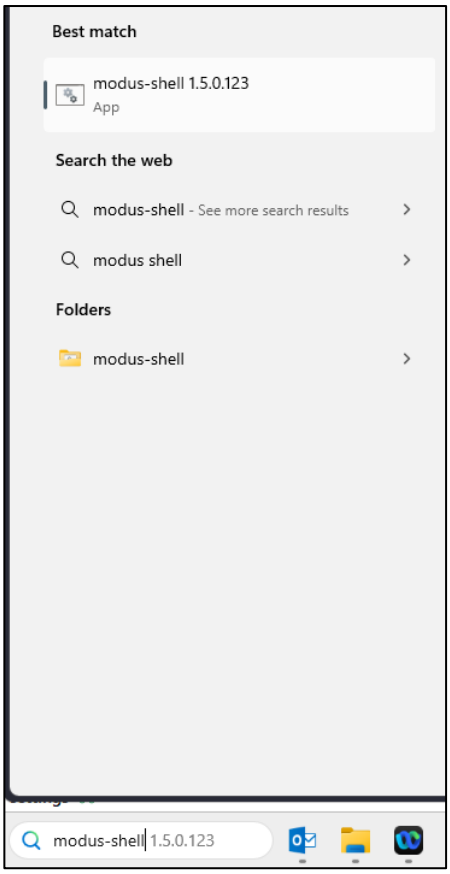
The IAR Embedded Workbench and Keil µVision IDEs have their own internal build system. If you are using one of those IDEs, you probably will want to stick with the IDE's build system. However, other make-based commands that you will see later such as `make library-manager` and `make device-configurator` can still be run from the command line on those applications.

2.2 Using the command line interface (CLI)

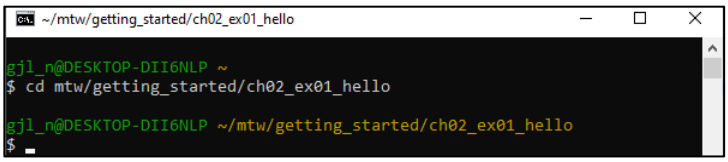
Each tool in ModusToolbox™ can be run using a command line interface that interacts directly with the make system for users that don't want to be constrained to working within an IDE.

2.2.1 Windows

To use the CLI, you need a shell that has the correct tools (such as git, make, etc.). The ModusToolbox™ ecosystem includes "modus-shell" which is based on Cygwin and ensures all settings and variables are correct for working with ModusToolbox™ tools. The shell is in the installation directory under *ModusToolbox/tools_<version>/modus-shell.* It is listed in the **Start** menu under **ModusToolbox <version>** or discoverable by typing `modus-shell` in the Windows search box until it appears in the list.



Once you start the shell, it will look like the following image. You will need to switch to the directory containing the ModusToolbox™ application that you want to operate on before running any command line operations.

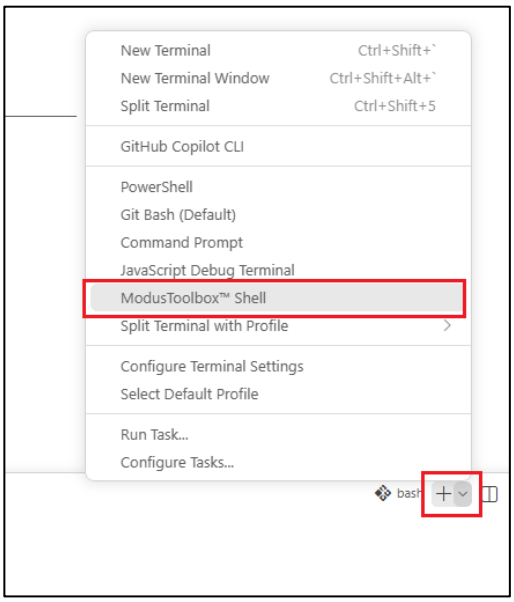


2.2.2 macOS / Linux

To run the command line on macOS or Linux, just open a terminal window and change to the application's root directory.

2.2.3 Integrated Terminal Panel

If you are using Infineon ModusToolbox™ for VS Code, there is an integrated Terminal panel at the bottom of the editor window. To open a new terminal instance, select **Terminal > New Terminal** from the menu (or press **Ctrl+`** on Windows, Linux, or macOS). After opening the terminal, select or navigate to the application's root directory before running any make commands. On the Windows operating system, you should use the modus-shell (Cygwin) terminal rather than the standard Windows command prompt or PowerShell, because modus-shell provides the tools required by ModusToolbox™ (such as make and git) with the correct environment settings. On macOS or Linux, a standard command terminal will work. The image below shows ModusToolbox™ Shell selected in the VS Code terminal.



2.2.4 Make targets (commands)

In order to have the command line commands work, you need to be at the top level of a ModusToolbox™ application or project where the *Makefile* is located.

The following table lists a few helpful make targets (i.e. commands). For more information, refer to the ModusToolbox™ tools package user guide.

Make command	Description
`make help`	This command will print out a list of all the make targets. To get help on a specific target, type `make help CY_HELP=<make target>`.
`make getlibs`	Process all the *mtb* files (or *lib* files) and bring all the libraries into your project. If you receive a shared application from someone, this is typically the next step.
`make build`	Build your project.
`make program` [Error! Bookmark not defined.]	Build and program your project.
`make qprogram` [Error! Bookmark not defined.]	Program without building.
`make device-configurator`	This command will open the Device Configurator.
`make get_app_info`	Prints all variable settings for the app.
`make get_env_info`	Prints environment information such as the tool versions being used.
`make printlibs`	Prints information about all the libraries including Git versions.

The help make target by itself will print out top level help information. For help on a specific command or variable use `make help CY_HELP=<command or variable>`. For example:

```
make help CY_HELP=build
```

```
make help CY_HELP=TARGET
```

```
$ make help CY_HELP=build
Tools Directory: C:/Users/follettcj/ModusToolbox/tools_3.6
Searching installed tools in progress...
Searching installed tools complete

Topic-specific help for "build"
  Brief: Builds the application.

The build process involves source auto-discovery, code generation, prebuild, and postbuilds. For faster incremental
builds, use the "qbuild" target to skip the auto-generation step. For multi-core applications, running this target b
uilds all projects in the application, and generates a combined hex file.
```

When running a command that takes significant time - such as build or debug – we highly recommended using a make option to allow multiple parallel jobs. For example, you can use `"-j8"` to specify eight parallel jobs should be run at a time. For example:

```
make -j8 build
```

Note: *The default configuration for building in Microsoft Visual Studio Code is setup to run 8 jobs (-j8).*

Note: *It is not advisable to use -j8 when using the program operation from the command line, especially for multi-core applications, because it can lead to build and program operations occurring in the wrong order. In that case, it is better to do a separate build using -j8 followed by qprogram. For example: make -j8 build; make qprogram.*

2.2.5 Make variables

The operations that `make` performs depends on many variables. These can either be set in the *Makefile* or they can be specified on the command line. A value set on the command line will override a setting for the same variable in the *Makefile*.

As an example, the following command line entry sets the `CY_HELP` variable to a value of `build`. This controls what the `make help` command does.

```
make help CY_HELP=build
```

There are many different make variables as you will see throughout the rest of this class. They control things like what to include or exclude in a build, toolchain, compiler flags and linker flags.

2.2.6 Accessing tools from the command line

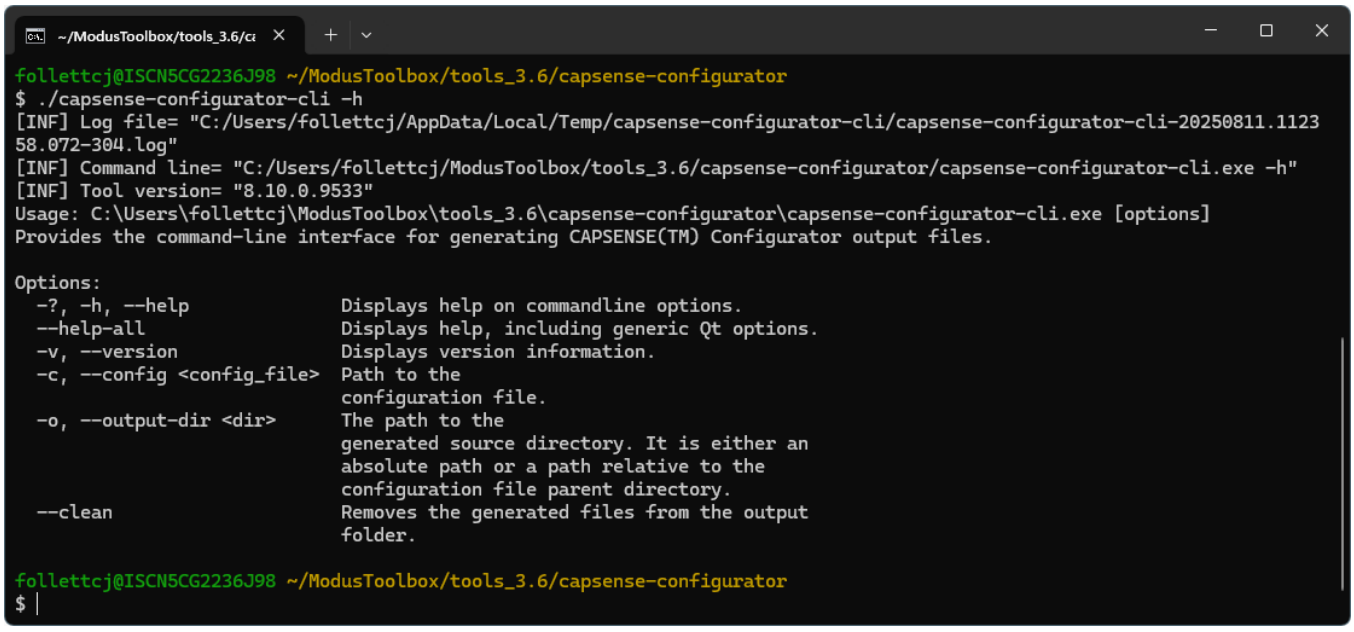
Most of the tools can be launched from an application's directory where the *Makefile* is located. Use `make` along with the executable name of the tool. A couple useful examples:

- `make device-configurator` launches the Device Configurator and opens the application's configuration file.
- `make library-manager` launches the Library Manager and opens the application's library settings.
- `make bsp-assistant` launches the BSP Assistant and opens the application's active BSP.

As described earlier, all tools can also be run directly from the tool's directory. You can use the **Start** menu entries or the search box on Windows.

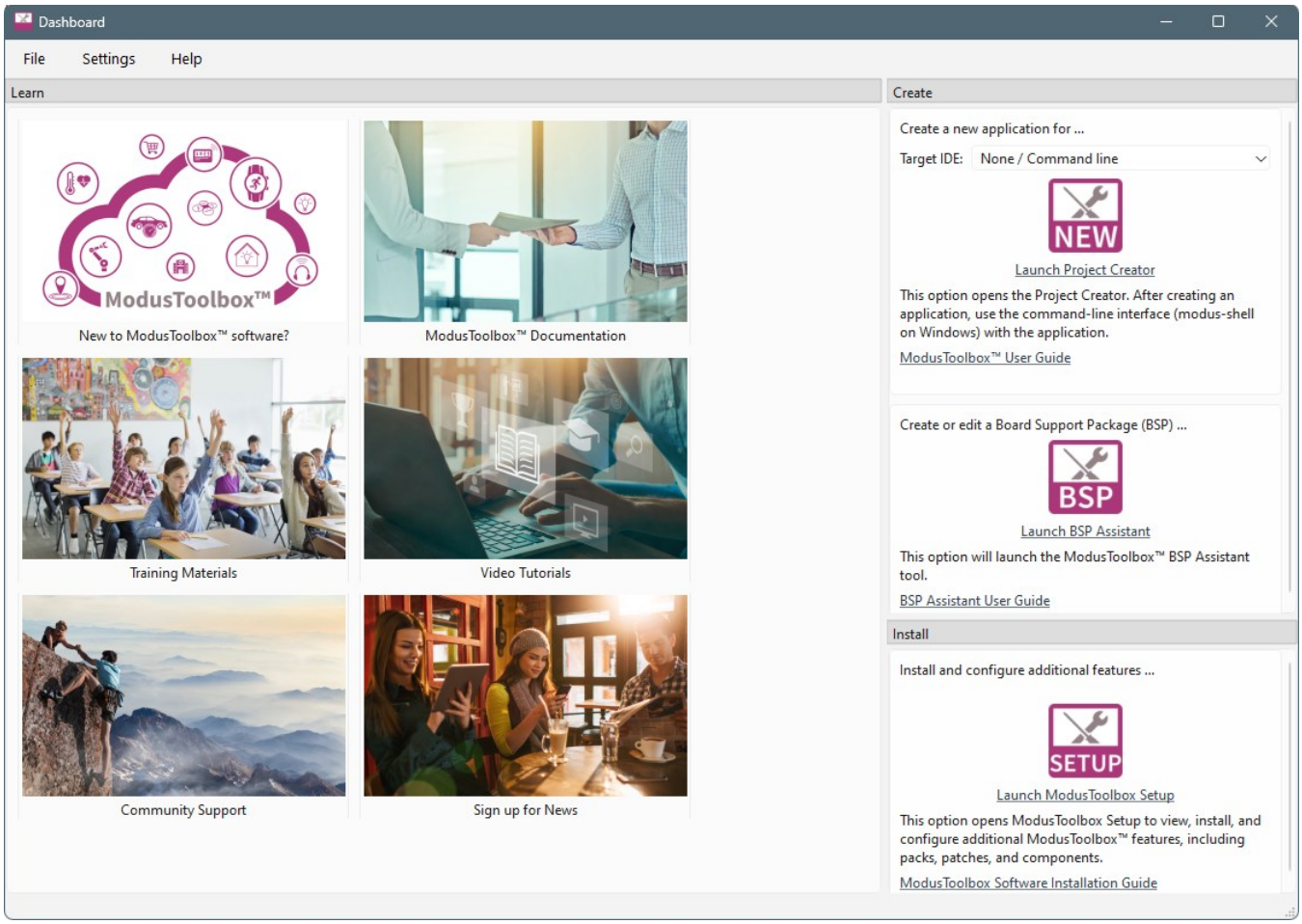
For many tools, there is a command line (i.e. non-GUI) version that can be useful for scripting/automation. The `-h` option shows help for the CLI tools. For example:

```
./capsense-configurator-cli -h
```



3 Dashboard

When you first install the ModusToolbox™ tools, the Windows installer gives you the option to launch the Dashboard tool. You can also launch this tool at any time from the Windows start menu, search box, or from the installation directory on any OS. The Dashboard looks like this:



On the left side, there are handy links to getting started information, documentation, training material, instructional videos, the community, and a link to sign up for ModusToolbox™ specific news feeds.

On the right side, there are panels to assist you in creating new applications and for creating or editing board support packages (BSPs). The dashboard can be used to launch Project Creator for the IDE of your choosing.

4 Windows Subsystem for Linux (WSL)

Due to the ModusToolbox™ architecture and the way that Windows handles files, the performance for compute intensive operations such as `build` can take longer than desirable on Windows. However, there is a way to circumvent that problem by running ModusToolbox™ tools using Windows Subsystem for Linux (WSL). This allows you to open an Ubuntu shell on your Windows machine and install/run ModusToolbox™ in that environment.

There is some initial setup required but once that's done it is relatively easy to use and the performance improvements can be quite drastic.

Application Note AN238846 describes the process to set up and use ModusToolbox™ in WSL.

5 Application creation

The first step you need to follow to start using ModusToolbox™ tools is to create an application.

Note: You can create a project using the VS Code extension. Refer to the Visual Studio Code for ModusToolbox™ user guide for more information.

5.1 Project Creator tool GUI

One way way to create an application is to use the Project Creator tool. It requires two pieces of information: (1) a starting BSP that specifies the hardware configuration; and (2) a starting template application.

The starting BSP can be an Infineon standard BSP, a custom BSP that you have previously created, or a BSP that is created on the fly by specifying the device(s) that the BSP contains – including the MCU and optional companion device such as a Bluetooth® or Wi-Fi module.

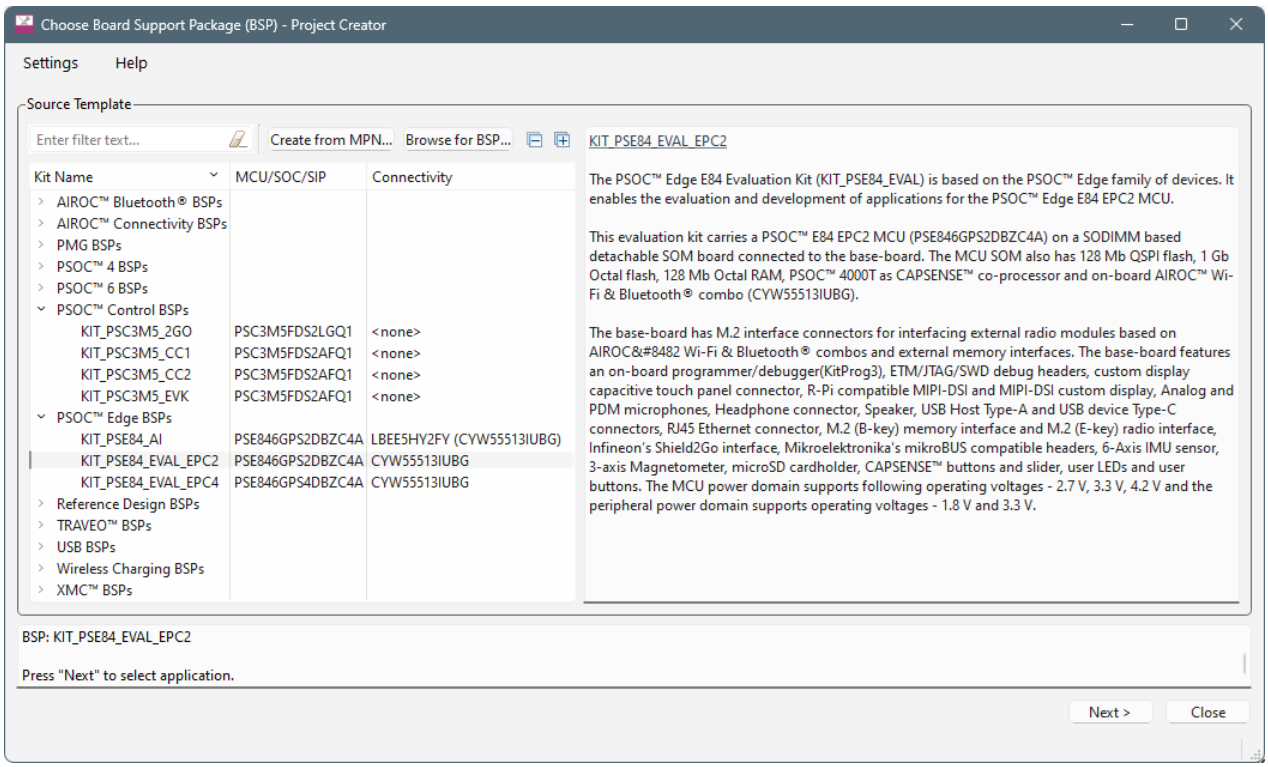
Note: This section describes creating a new application from scratch using a template application. The process to import an existing application into an IDE or to share an existing application with another team member is different. That process is covered in section 14.

In Windows, it is in the **Start** menu or you can enter **project-creator** in the Search Box. On all systems, you can run it from the ModusToolbox™ tools installation directory. The default location is *~/ModusToolbox/tools_<version>/project-creator/project-creator.exe*. As described above, the Dashboard tool can also be used to facilitate launching Project Creator.

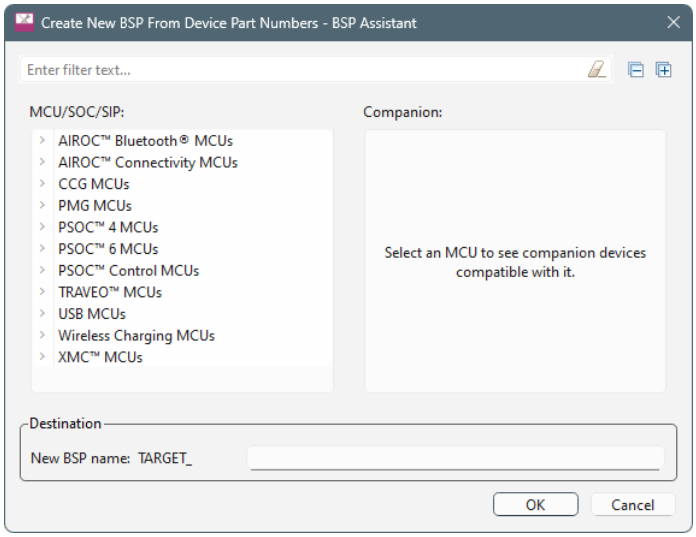
The first thing the Project Creator tool does is read the configuration information from Infineon's GitHub site so that it knows all the BSPs and code examples that are available. That information is stored in *manifest* files which we will discuss in section 15.

Note: You can also create projects using local content. See section 17.3 for details.

Once the manifests have been read, the first window asks you to select the target BSP. The target BSPs are categorized by device family, as shown below.



Note: *The **Create from MPN** button launches the BSP Assistant tool to create a custom BSP on the fly by specifying the manufacturing part number (MPN) for the MCU and optionally a companion device such as a Bluetooth® or Wi-Fi module on your board. The new BSP will be added to the list of BSPs that you can select.*



Note: *The **Browse for BSP** button allows you to add a custom BSP that you previously created to the list of BSPs.*

Select a BSP from the list (either standard or custom) and click **Next >**.

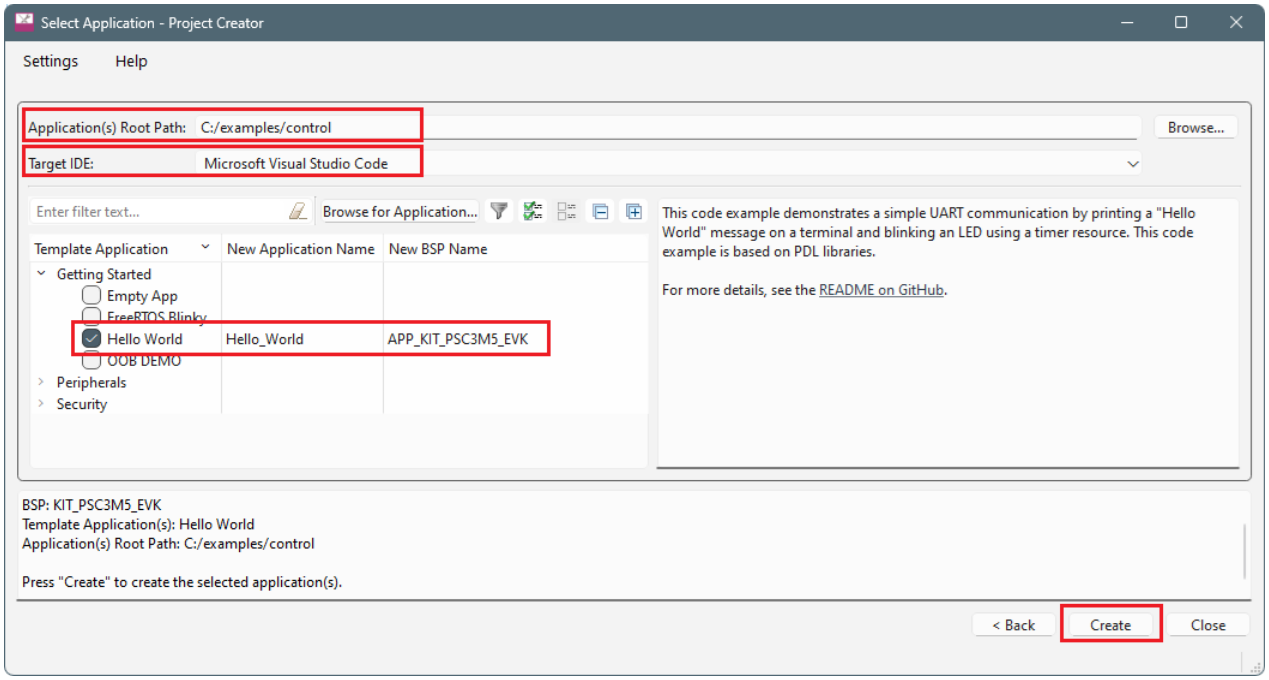
You will be presented with all the template code examples supported by the BSP you chose. Pick an application to start from and give it a name. The applications are arranged by categories to make it easier to find what you are looking for. Only applications that are valid for the BSP you chose in the previous step will appear.

Note: *If you selected a custom BSP in the previous step, all applications that are supported by the device(s) on your board will be shown. Since Project Creator doesn't know what other hardware is available on your board (e.g. LEDs, buttons, etc.) and what names you chose for them, some applications may not work as-is due to the available hardware resources or names.*

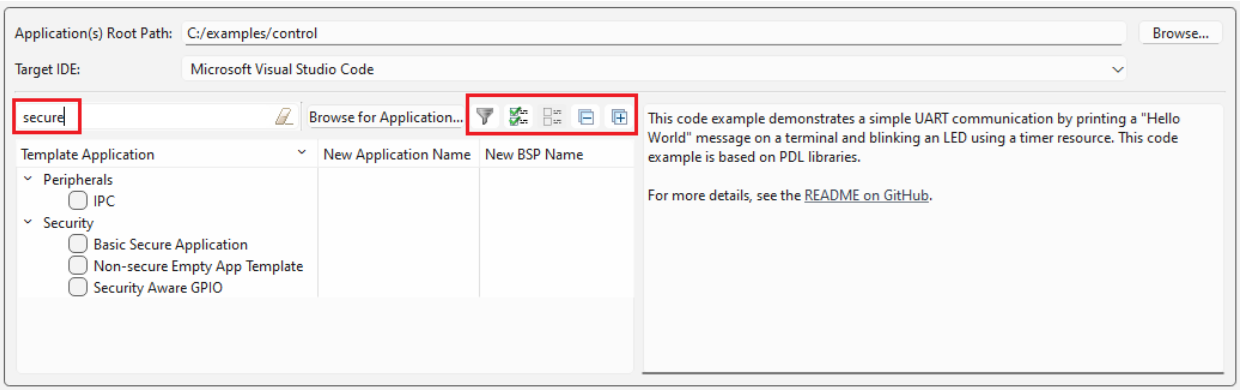
You can specify a different **Application Root Path** if you don't want to use the default value. A directory with the name of your application(s) will be created inside the specified Application Root Path. The value for the Application Root Path is remembered from the previous invocation, so by default it will create applications in the same location that was used previously.

The **Target IDE** selection lets you decide if you want files to be created for use with an IDE. Currently it supports Microsoft Visual Studio Code, Eclipse for ModusToolbox™, IAR Embedded Workbench, and ARM Keil™ µVision™. Select Microsoft Visual Studio Code to generate the files needed to open the application in VS Code. If you don't select an IDE at this step, you can generate the required files later using the command line. More details on this feature will be available later in the chapter.

Note: *If you choose IAR Embedded Workbench, the project will be set up to use the IAR toolchain, but the application's Makefile will not be modified. Therefore, builds inside IAR Embedded Workbench will use the IAR toolchain while builds from the command line will continue to use the toolchain that was previously specified in the Makefile. You can edit the Makefile's TOOLCHAIN variable if you also want command line builds to use the IAR toolchain.*



You can use the filter text box to enter a string to match from the application names and their descriptions. For example, if you enter *secure*, you may see a list like this. (The exact list will change over time as new code examples are created). Only categories containing matching applications will be listed.

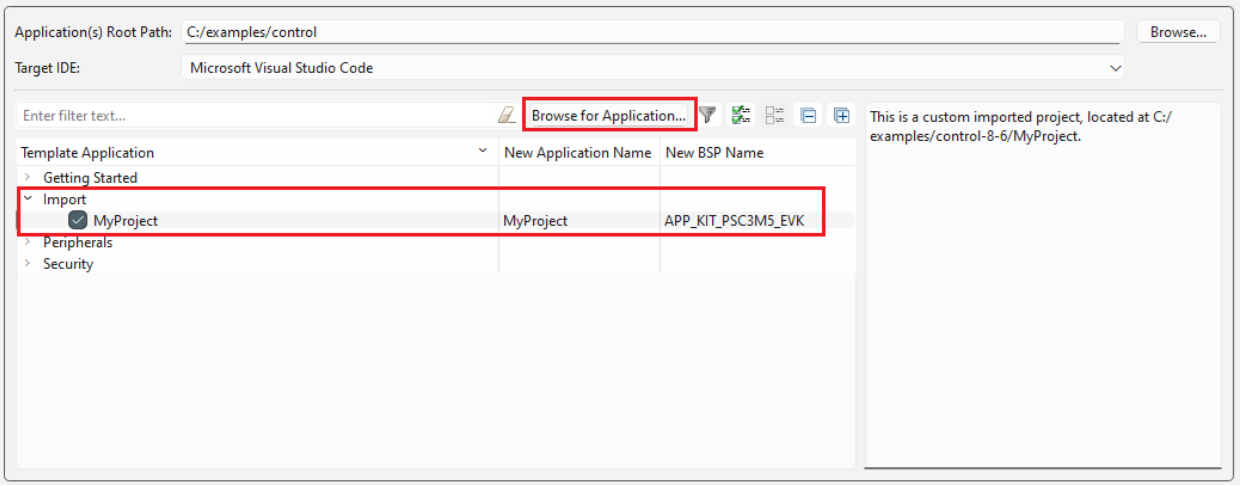


If desired, you can select all the applications in the search result by using the **Select All** button or you can unselect them with the **Unselect All** button. You can also expand or collapse all categories with the **Expand All** and **Collapse All** buttons.

The **Filter** button can be used to show only the applications which are currently checked. This can be useful if you have a large number of applications checked and want to view them all listed together.

If you want to start with your own local template application instead of one of the code examples that we provide, click the **Browse for Application** button. This allows you to browse to an application or application template on your computer and then create a new application based on it. Make sure the path you select leads to the directory that contains the Makefile, otherwise your application creation will fail.

Once you select an application directory, it will appear in the **Import** category.



Note: *The existing project can be located anywhere on your system. It does not need to already have VS Code files (such as a .code-workspace file). At the very least, it needs a Makefile and source code files, however it may also contain other items such as configurator files and mtb files (which are a mechanism to include dependent libraries in an application). If VS Code files are not present, you can generate them using the make vscode command.*

Before clicking the **Create** button, you can change the name of the application and the name of the BSP if you desire. You may have to double-click in the appropriate name box to enable editing, Then, just enter the desired names.

Once you have selected the application you want to create (whether it's an Infineon code example or one of your own applications), and specified the names you want to use, click **Create** and the tool will create the application.

The tool runs several operations including `git clone` and `make getlibs`. The project is saved in the directory you specified previously. When finished, click **Close**.

5.2 Project Creator CLI

The Project Creator tool described above also has a command line version that you can run from a shell (e.g. modus-shell). It is in the same directory as the Project Creator GUI (*~/ModusToolbox/tools_<version>/project-creator*) but the executable is called *project-creator-cli.exe*. You can run it with no arguments to view the different options available. For example, you can run it to see a listing of all the available BSPs:

```
project-creator-cli --list-boards
```

You can also see all the available applications for a given BSP:

```
project-creator-cli --list-apps <BSP_Name>
```

In order to use the Project Creator in CLI mode to create an Hello World application for the KIT_PSC3M5_EVK kit and place it in the user's *mtw/* directory with a name of *my_app*, the command is:

```
project-creator-cli
    --board-id KIT_PSC3M5_EVK
    --app-id mtb-example-ce240510-hello-world
    --target-dir "~/mtw"
    --user-app-name "my_app"
```

Note: *If you are using a tilde (~) to specify the user's home directory, double-quotes must be placed around the target directory name.*

Other arguments are available to create an application using a local custom BSP (--board-path) or a local custom application (--app-path) if desired. This is equivalent to the **Browse** buttons in the GUI.

5.3 Template processing

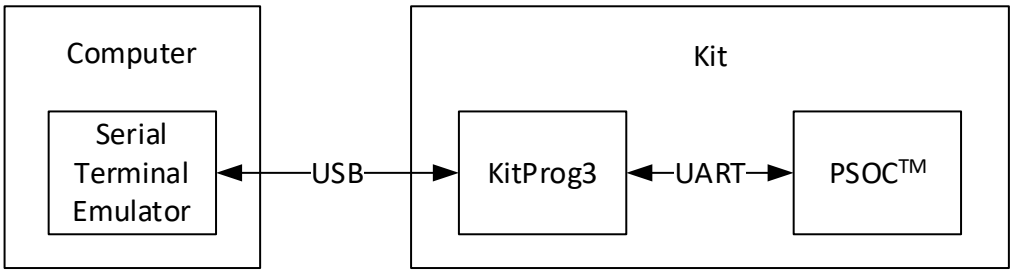
Some code examples have a directory called *templates* that contains files to override the same ones in the BSP. Most often this is done when the application needs a different device configuration or different linker scripts for multi-core applications. When an application is created from such a code example, the files from the *templates* directory for the target BSP are copied into the BSP in the *bsps* directory. This allows code examples to change the default BSP behavior without having to supply a complete custom BSP. If the BSP is not located in the *bsps* directory, the *templates* directory will be ignored.

The *templates* directory is excluded from the build process by including it in a *.cyignore* file or the `CY_IGNORE` variable in the *Makefile*.

6 Serial Terminal Emulator

Most Infineon kits have an integrated programming device (such as KitProg3 on PSOC™ kits) which also have a UART to USB interface. This is very useful for printing messages from the firmware onto a serial terminal window on your computer. For example, with PSOC™ this can be done either using the *retarget-io* library with the standard C `printf` function or using the UART API directly. Either way, the UART signals from the PSOC™ go to the KitProg3 device on the kit. The KitProg3 device converts the UART messages into USB and sends them to the PC over the USB connector. On the PC side, a serial terminal emulator is used to display the messages.

Note: *The UART is bi-directional, meaning that it allows you to type into the serial terminal emulator on the PC and send those messages to the PSOC™.*



There are various serial terminal emulators available for use. On Windows and Linux, a program called *PuTTY* is typically used. On MacOS, you can use a program called *screen*. If you are familiar with any other serial terminal emulator, feel free to use that instead.

6.1 Determine the Port

On Windows you can find the correct serial line in the device manager under **Ports (COM & LPT)** as "KitProg3 USB-UART." It will be something like COM91.

On Linux, issue the following in a command window. Your kit will be something like */dev/ttyACM0*.

```
ls /dev/tty*
```

On MacOS, issue the following in a command window to look for a USB serial device. Your kit will have "usb" in the name.

```
ls /dev/tty.*
```

On both Linux and MacOS, a quick way to find the port is to use the above ls command with the kit connected and disconnected. Observe from the results which device only appears when the kit is connected.

6.2 PuTTY (Windows and Linux)

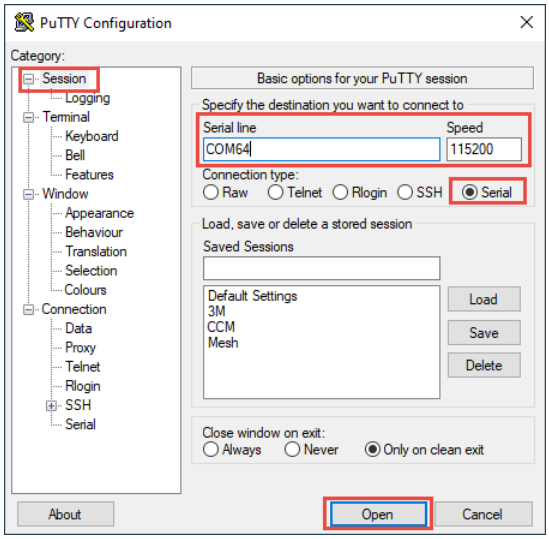
PuTTY for Windows can be found online. Once it is downloaded, you just run the executable `putty.exe` – it does not need to be installed.

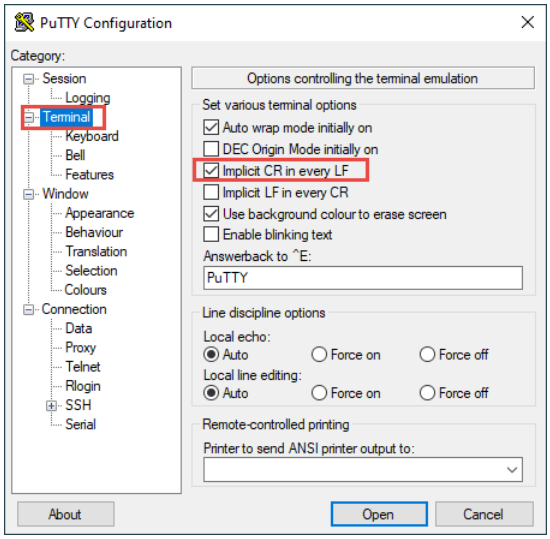
For Linux (Ubuntu), PuTTY can be installed by running the following commands from a command window.

```
sudo apt-get update
sudo apt-get install -y putty
```

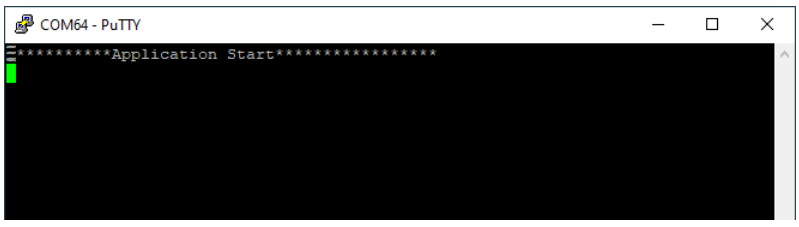
When putty starts up, first go to the **Session** settings. Click the "Serial" button if it is not already selected. Set the correct Serial line and set the Speed to 115200. Other settings such as flow control and parity can be found in the **Serial** settings but the defaults should be correct. Finally, in the **Terminal** settings, check the box for "Implicit CR in every LF." This step ensures that printing the `\n` character will send both a line feed and a carriage return. Once you have everything set the way you want it, click Open.

*Note: Before opening the terminal, you can use the **Save** button to save your settings as the default settings or you can save to a new name if you want to keep more than one set. Just click on the session you want to save (e.g., Default Settings) or type a new name and then click **Save**.*

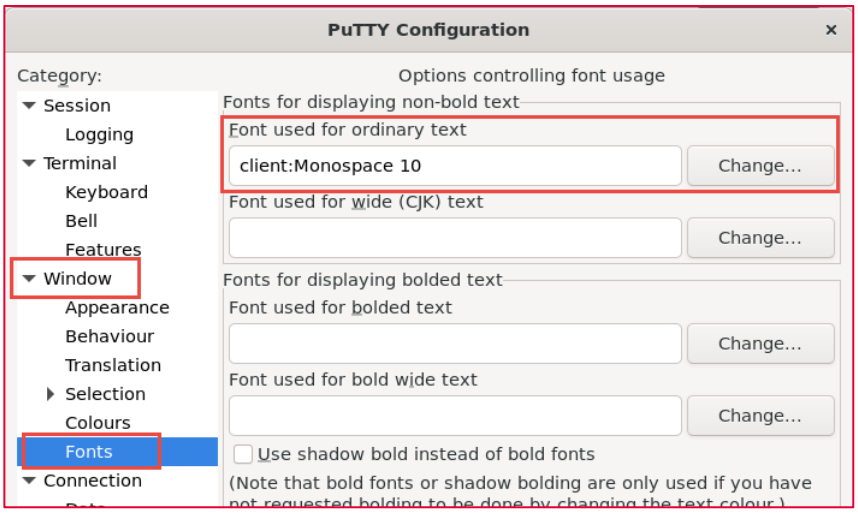




Once it is running, you can right-click on the top banner to show a list of useful commands. You can change settings, clear the screen, and even restart the session of the kit if it was disconnected.



*Note: On Linux, if you see the error message **PuTTY: unable to load font "server:fixed"** when you try to open the connection, go to **Window > Fonts > Font used for ordinary text** and click **Change…** to select a different font from the list, such as **client:Monospace 10**. It is useful to save this to the settings from the Session tab as described above so you don't have to repeat each time.*



Note: *On Linux, you can run commands such as `putty` in the background by adding an ampersand (`&`) at the end of the command. This allows you to keep putty running while still using the command terminal for other tasks. For example: `putty &`*

Note: *If you forget to put a program in the background when you first start it, just enter `Ctrl-z` in the terminal window and then enter the command `bg` to put the last process in the background.*

6.2.1 Screen (MacOS and Linux)

For MacOS and Linux, the `screen` program is a good serial terminal emulator.

Once you find your kit's port as explained in 6.1, use the following command to start `screen`:

```
screen /dev/tty<your_device> 115200
```

You can disconnect from the terminal and leave it running in the background by pressing `Ctrl-a Ctrl-d`. To switch back to it, use `screen -r`.

When you want to kill the terminal window, use `Ctrl-a Ctrl-k`.

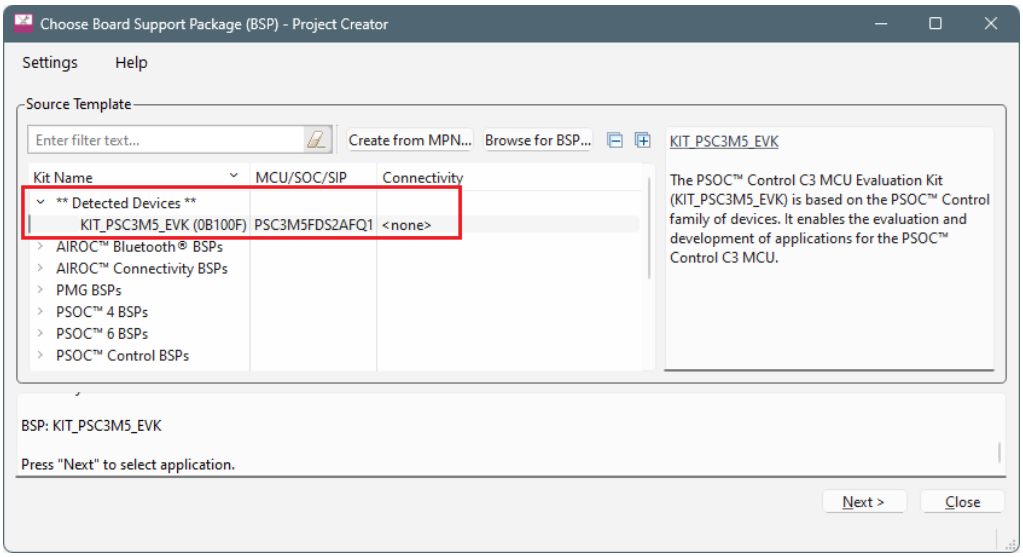
7 Exercises (part 1)

For the following exercises, the KIT_PSC3M5_EVK kit is shown, but you can choose to use any available BSP to explore the family or solution of your choice. The exact steps and starting application that you use may be different than what is shown in the following exercises, but the concepts will be the same.

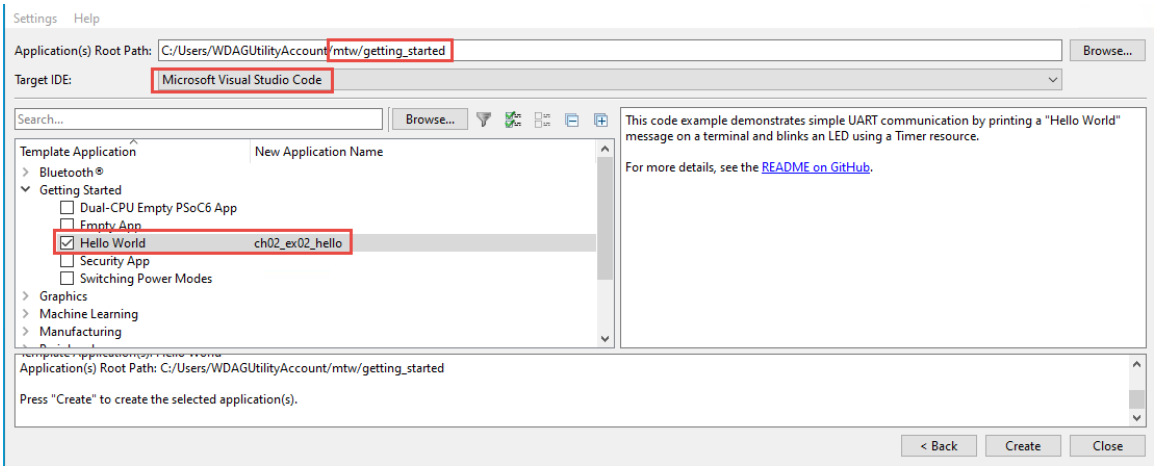
Exercise 1: Create, build, program and debug a Hello World application in VS Code

For this exercise, we will create, build, and program an application using VS Code.

1. Run the Project Creator tool.
2. Once Project Creator launches, select a kit. In this example, the kit is attached and selected by default.

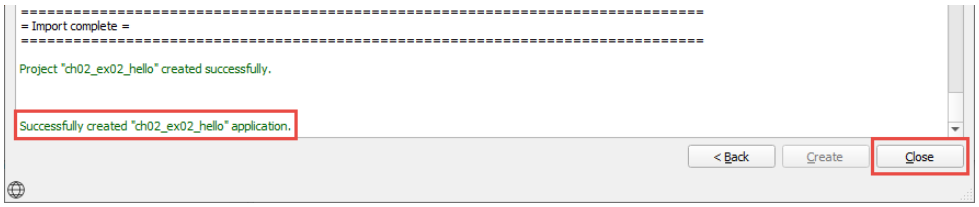


3. Click **Next** to proceed.
4. On the Application page, select the *Hello World* application and change the name to **ch02_ex02_hello**. Change the **Target IDE** to Microsoft Visual Studio Code, if needed. We'll leave the existing Application root path so that it will go in the same workspace as the previous exercise, but you can change that if you wish.



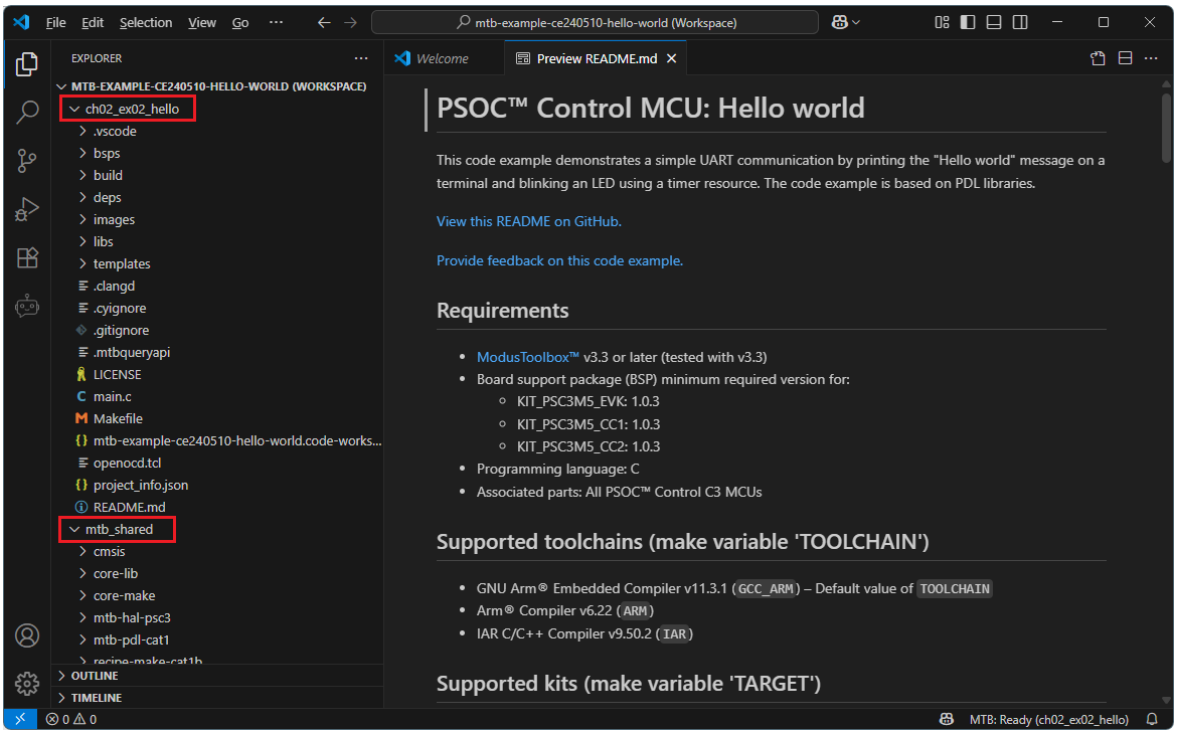
Note: *If you forget to select the **Target IDE** when you create the application, remember that you can run `make vscode` from the command line once application creation is complete.*

5. Once the application has been created, click the **Close** button, if needed.



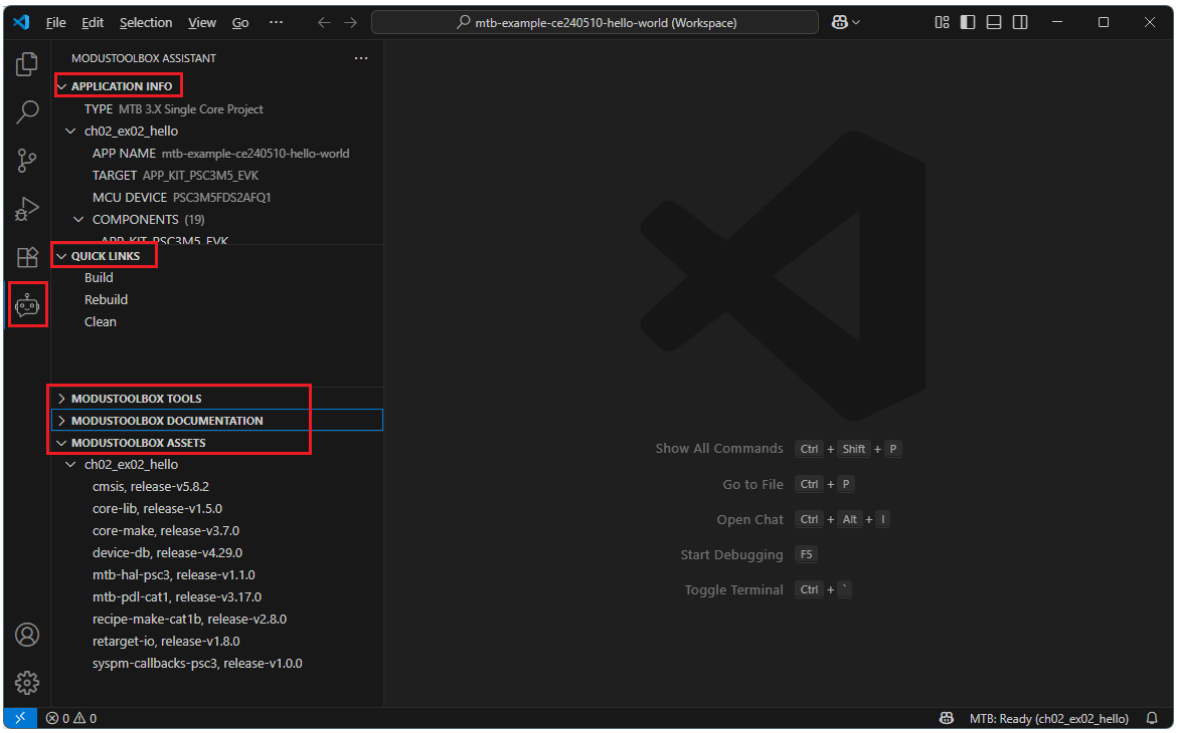
Note: *If launched from VS Code, the Project Creator tool will close automatically if there were no errors or warnings.*

6. Open the application in VS Code and open the workspace. If you open it correctly, you will see the application directory and the *mtb_shared* directory.

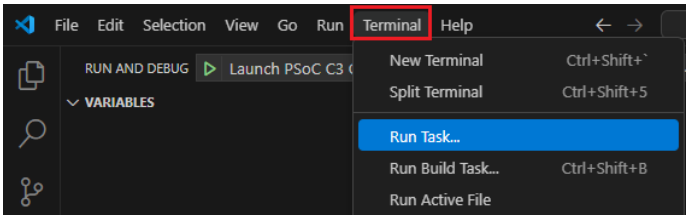


7. The *README.md* file should be open. If not, right-click on it and select **Open Preview** to see the formatted Markdown file.

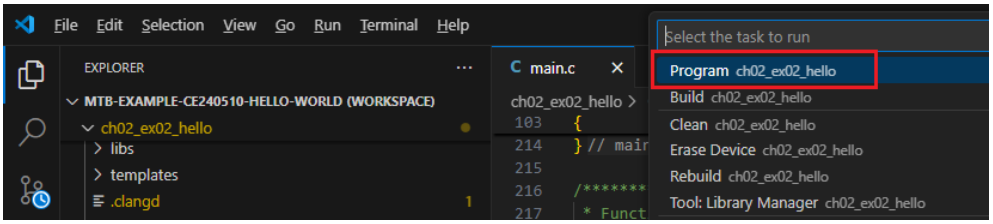
8. If installed, click on the icon for the ModusToolbox™ Assistant extension. Here you can explore all of the different tools, documenation and information available to you.



9. To program, first select **Run Task** from the **Terminal** menu.



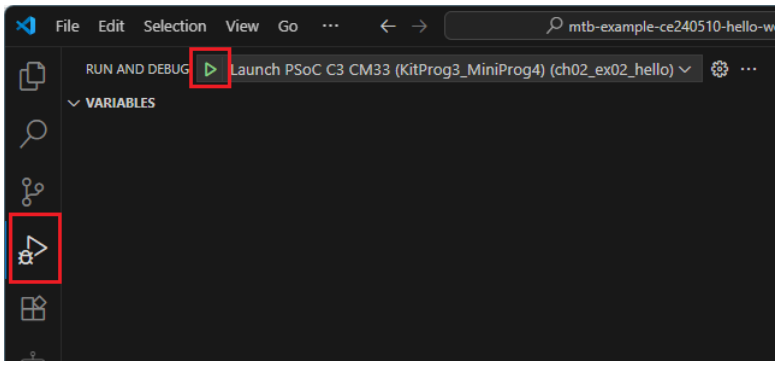
Then, select the **Program** item from the "Select the task to run" prompt.



At the bottom of the screen, you will see build and program messages in the terminal window. When it finishes, you should now see the blinking LED.

You can also open a serial terminal window to see messages. Refer to the *README.md* file for the code example instructions.

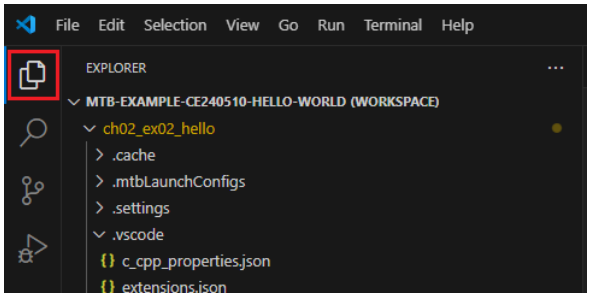
10. To debug, first select **Run & Debug** icon on the left, and then click the green triangle next to the **Launch** configuration.



11. Start/pause execution, single step through the code, and add a breakpoint. See the VS Code user documentation for additional information on debugging. Click the **Stop** button when done.



12. To go back to the project explorer window view, click the **Explorer** icon on the left side.



Exercise 2: Build and program using the Command Line Interface

For this exercise, you will build and program an existing application using the CLI on one of your existing applications.

1. Start modus-shell (Windows) or open a command terminal (other platforms).
2. Switch to the directory for the exercise 1 application.
3. Enter the command `make help` to see a list of possible commands.
4. Enter the command `make qprogram` to program the board using the existing hex file without rebuilding first.

8 Board Support Packages (BSPs)

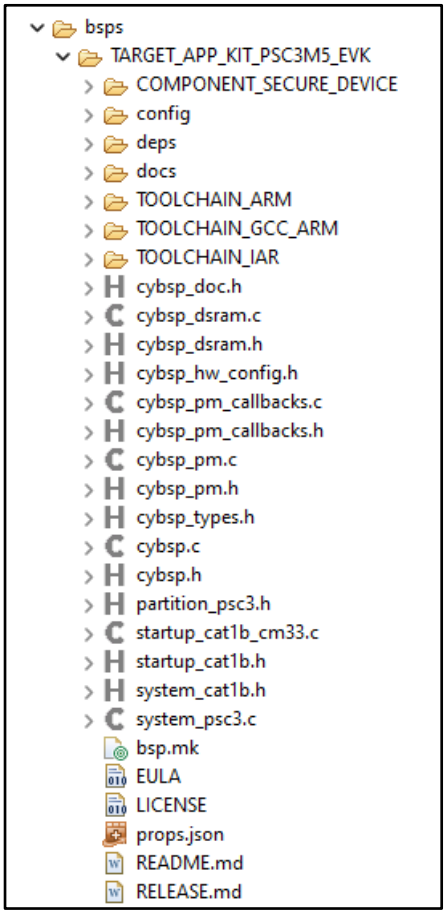
Each project is based on a target set of hardware. This target is called a "Board Support Package" (BSP). It contains information about:

- The chip(s) on the board, how they are programmed, and how they are connected
- Device configurations such as clocks, pins, and on chip-peripherals like PWMs, and ADCs
- The peripherals on the board such as LED, buttons, and sensors
- What the device pins are connected to on the board
- What libraries the BSP depends on

A BSP directory starts with the keyword "TARGET" and can be seen in the *bsps* directory of the application because the BSP is local to the application. In fact, when you create an application or add a new BSP to an application, the BSP that you select is copied into the application's *bsps* directory and its Git information (if any exists) is removed. At that point, the application "owns" the BSP for that application. This allows you to modify the BSP configuration for an application without dirtying a Git repo or affecting all other applications in the same workspace.

8.1.1 BSP directory structure

The following image shows a typical directory structure for a PSOC™ Control MCU BSP. The files



The various directories and files in a BSP include the following, and they vary by device family:

Directory/File	Description
bluetooth/	Contains information for a companion Bluetooth® device, if present, that is used in conjunction with the onboard MCU. This includes pin connections between the devices, and transport mechanism details.
COMPONENT_xxx	For some BSPs, this contains start-up code and linker scripts. For other BSPS, this includes code specific to a device for various components, such as security.
config/	Contains BSP configuration files. These are edited by using the appropriate configurator. For example, *design.modus* is edited using the Device Configurator. Contains the *cyreservedresources.list* file used to reserve resources so that the user cannot change settings or attempt to use them for other purposes in the Device Configurator. This is typically only used for devices that require specific UDB, routing, or clocking resources to implement an advanced feature. Contains the *GeneratedSource* directory that is created by the configurators.
deps/	Contains *mtbx* files for libraries that the BSP requires as dependencies. These files are generated during application creation using the dependency information contained in a manifest file. Manifest files are explained in more detail in section 15.
docs/	Contains HTML based documentation for the BSP.
TOOLCHAIN_xxx/	Startup code and linker scripts for all supported toolchains.
.cyignore	Used to exclude directories from the build if necessary for this BSP.
bsp.mk	Defines the DEVICE and other BSP-specific make variables such as `COMPONENTS`.
cybsp.h *cybsp.c*	These files contain the API interface to the board's resources. You need to include only *cybsp.h* in your application to use all the features of a BSP. Call the `cybsp_init` function from your code to initialize the board (that function is defined in *cybsp.c).*
cybsp_doc.h	Used to generate documentation on the BSP's resources. It is not used for any functional purpose.
cybsp_dsram.h *cybsp_dsram.c*	Contains initialization code for deep sleep RAM.
cybsp_hw_config.h	Contains common aliases which are expected by code examples.
cybsp_pm.h *cybsp_pm.c* *cybsp_pm_callbacks.h* *cybsp_pm_callbacks.c*	Contains power mode callbacks and their registration. These callbacks prepare resources when entering or exiting various power mode, e.g. preparing clocks for entering deep sleep.
cybsp_types.h	Contains macros for using various board peripherals. For example, `CYBSP_LED_STATE_ON` and `CYBSP_LED_STATE_OFF` are set to the appropriate values based on the LED hardware on the board (e.g. active low or active high).
gpio_xxx.h	Device-specific information on pins and HSIOMs. On older BSPs, this information comes from the HAL and PDL.
EULA/LICENSE	End-user license agreements.
mtb_xxx_config.h	Exposed configuration options for a middleware or library asset. These files are owned by the user and their contents can be updated directly to configure the asset.
partition_xxx.h	Contains memory region address and sizes.
props.json	Contains BSP information including but not limited to capabilities that the BSP supports, version information, category for the Project Creator and Library Manager, and BSP capabilities.

Directory/File	Description
README.md	Contains BSP documentation in Markdown format.
RELEASE.md	Includes BSP release notes.
startup_xxx.h/startup_xxx.c *system_xxx.h/system_xxx.c*	Contains device system header information for MCU BSPs. The file name will be different for other device families.

8.2 BSP dependency version locking

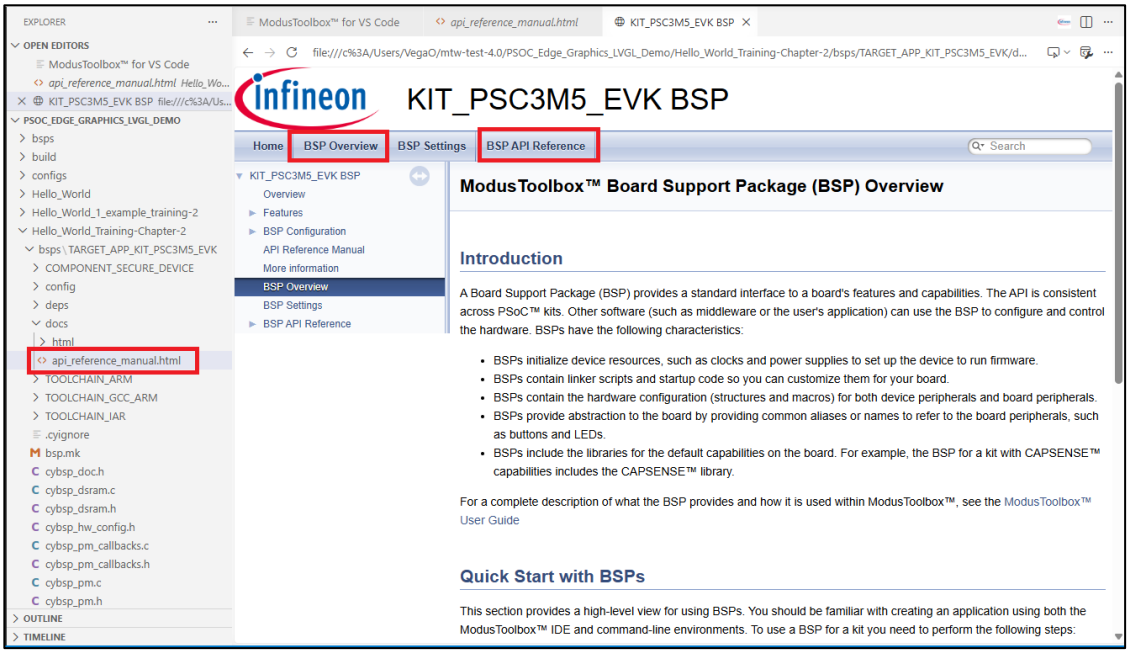
The dependencies of a BSP are stored in *mtbx* files in the BSPs *deps* directory. When a BSP is created inside an application, the version of each dependency is locked to a specific version so that it does not change in that application unless the user explicitly requests a change. To change the version of a dependency inside a BSP, use the BSP Assistant - which will be discussed in a minute. To change the version of a dependency inside an application without affecting the BSP itself, use the Library Manager.

8.3 BSP names

When creating an application from a manifest-based BSP, the default name of the in-app BSP created in the application will use the same name with a prefix of *APP* (e.g. *TARGET_APP_KIT_PSC3M5_EVK*). When creating an application using a local custom BSP via the **Browse for BSP** mechanism, the default name of the in-app BSP created in the application will be the same as the name of the provided BSP. However, you can change the name of the BSP that will be created inside the app during application creation using Project Creator or later on using the Library Manager.

8.4 BSP documentation

Each BSP provides HTML documentation that is specific to the selected board. It also includes an API Reference and BSP Overview. This documentation is located in the BSP's "docs" directory. For example:



9 Application directory organization

A ModusToolbox™ application will contain one or more projects. In a simple application that uses only one CPU core, the application and the project will be part of the same directory. In a typical multi-core application, the top-level directory will be part of the application and it will have subdirectories containing the project for each core.

9.1 Single-core application

For applications that only use one CPU core, there is just one project. In that case, the top-level application directory and project directory are the same. There is usually another directory containing libraries that may be shared between applications.

The exact files and directories may be different for other applications, device families or solutions but the basic structure will be the same. The directories and files in the example application's project directory include:

Directory/File	Description
Binaries	Virtual directory that points to the *elf* file from the build. This directory may not appear until a build is done.
bsps	This directory contains the board support packages (i.e. BSPs) that are owned by the application. The BSPs in this directory are copies of the BSP selected during application creation and any BSPs added to the application using the Library Manager. They are no longer Git controlled libraries. Any edits done to these BSPs apply only to this application.
build	This directory contains build files, such as the *elf* and *hex* files.
deps	This directory contains *mtb* files that specify where the `make getlibs` command locates the libraries directly included by the project. Notice that libraries included via *mtb* files may have their own library dependencies listed in the manifest files. We'll talk more about dependencies, *mtb* files and manifests later in the chapter. As you will see, the source code for libraries either go in the shared repository (i.e. for shared libraries) or in the *libs* directory inside the application (i.e. for libraries that are not shared).
images	This directory contains artwork images used by the documentation.
libs	The *libs* directory contains source code for local libraries (i.e. those that are not shared) and *mtb* files for indirect dependencies (i.e. libraries that are included by other libraries). Most libraries are placed in a shared repo and are therefore not in the *libs* directory. The *libs* directory also includes a file called *mtb.mk* which specifies which shared libraries should be included in the application and where they can be found. *Note: As you will see, the libs directory only contains items that can be recreated by running `make getlibs`. Therefore, the libs directory should normally not be checked into a version control system.*
templates (not shown)	Some code examples have a directory called *templates* that contains files to override the default BSP configuration. See section 5.3 for details.
main.c	This is the primary application file that contains the application's code. Depending on the application, this file may have a different name or have multiple source code files. In more complex applications, the source code files might be in a subdirectory such as *source*.
LICENSE	This is the ModusToolbox™ software license agreement file.
Makefile	The *Makefile* is used in the application creation process. It defines everything that the ModusToolbox™ software needs to know to create/build/program the application.
README.md	Almost every code example includes a *README.md* file(in Markdown format) that provides a high-level description of the application.

9.1.1 Makefile

Various build settings can be set in the *Makefile* or can be specified on the command line to change the build system behavior. These can be "make" settings or they can be settings that are passed to the compiler. Some examples are:

Application type

For single core applications, the variable `MTB_TYPE` should be set to `COMBINED`. This indicates that the application and project are both located in the same directory since there is only one project that makes up the application.

Target

This variable sets the BSP that will be used for the build. A directory named `TARGET_<target variable value>` must be in the application's search path. If there is more than one directory that starts with `TARGET` in the search path, only the directory specified in the `TARGET` variable will be used.

In the example below, the BSP must be contained in a directory named `TARGET_KIT_PSC3M5_EVK`.

```
# Target board/hardware (BSP).
# To change the target, it is recommended to use the Library manager
# ('make modlibs' from command line), which will also update VS Code launch
# configurations. If TARGET is manually edited, ensure TARGET_<BSP>.mtb with a
# valid URL exists in the application, run 'make getlibs' to fetch BSP contents
# and update or regenerate launch configurations for your IDE.
TARGET=KIT_PSC3M5_EVK
```

Note: *The `TARGET` board should have an mtb file in the deps directory (if it is a standard BSP) and must be in the source file search path if it is in the shared location. Therefore, if you change the value of `TARGET` manually in the Makefile, you must add the mtb file for the new BSP and then run `make getlibs` to update the search path in the libs/mtb.mk file.*

Note: *`TARGET` is used in the launch configurations (program, debug, etc.) so if you change this variable manually in the Makefile, you must update or regenerate the Launch configs inside VS Code. Otherwise, you will not be programming/debugging the correct firmware.*

Given the reasons listed above, it is better to use the Library Manager to update the TARGET BSP because it makes all the necessary changes to the project. If you are working exclusively from the command line and do not want to run the Library Manager, you can regenerate the *libs/mtb.mk* file by running `make getlibs` once the appropriate *mtb* files are in place.

Build configuration

The build configuration sets compiler optimization settings. What each selection does is toolchain specific. If you choose Custom, you can manually set any compiler optimization that you want using the `CFLAGS` variable.

```
# Default build configuration. Options include:
#
# Debug -- build with minimal optimizations, focus on debugging.
# Release -- build with full optimizations
# Custom -- build with custom configuration, set the optimization flag in CFLAGS
#
# If CONFIG is manually edited, ensure to update or regenerate launch configurations
# for your IDE.
CONFIG=Debug
```

Note: *CONFIG is used in the launch configurations (program, debug, etc.) so if you change this variable manually in the Makefile, you must update or regenerate the Launch configs inside VS Code. Otherwise, you will not be programming/debugging the correct firmware.*

Components

The COMPONENTS variable is a way to enable code to be optionally included in a build. Any directory whose name is COMPONETS_<name> will only be included in the build if <name> is included in the COMPONENTS variable. Use spaces between names if you need more than one COMPONENT.

There is also a variable called DISABLE_COMPONENTS which will remove a directory from the build even if it is included elsewhere using the COMPONENTS variable. This can be used to override the default behavior of a BSP or library.

```
################################################################################
# Advanced Configuration
################################################################################

# Enable optional code that is ordinarily disabled by default.
#
# Available components depend on the specific targeted hardware and firmware
# in use. In general, if you have
#
#    COMPONENTS=foo bar
#
# ... then code in directories named COMPONENT_foo and COMPONENT_bar will be
# added to the build
#
COMPONENTS=

# Like COMPONENTS, but disable optional code that was enabled by default.
DISABLE_COMPONENTS=
```

Note: *When you add a library to an application, a COMPONENT is automatically created for the name of the library with the prefix MW. For example, if you add the capsense library to an application, a COMPONENT is created called MW_CAPSENSE.*

Including files and directories

The make system automatically searches through the application's directory hierarchy for source files to include in the build. However, there are several mechanisms to include other code if necessary, such as from custom libraries that are located outside the application.

There are three main variables that you can use. Each one can use a relative path and can have a list of more than one item separated by spaces.

SOURCES	Use this variable to specify a list of C/C++ source files to include in the build.
INCLUDES	Use this variable to specify a directory containing header files to include in the build. Only the specific directory is included – not sub-directories.
SEARCH	Use this variable to specify a directory to add to the auto-discovery path. This means that the directory and all of its sub-directories will be searched by the make system. SEARCH is the variable that is typically used with custom libraries that are outside the application's directory.

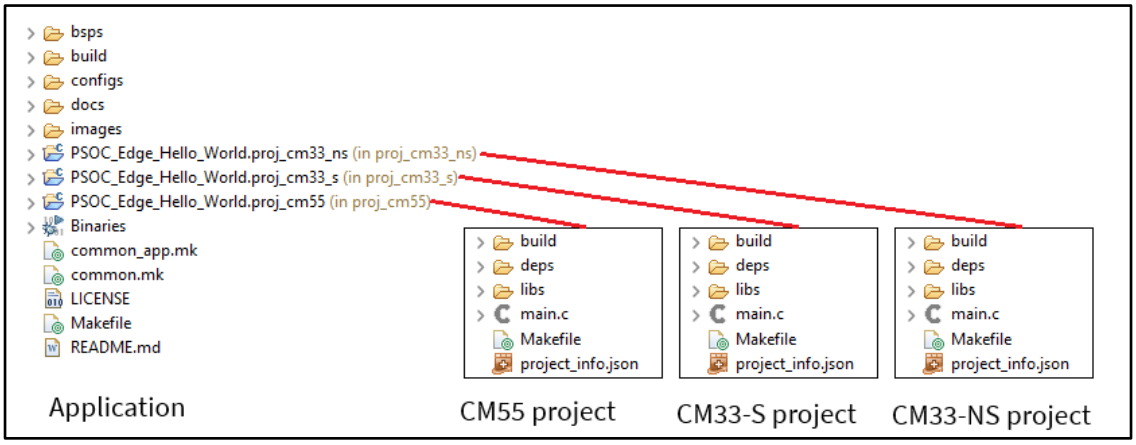
Note: *The `SOURCES` and `INCLUDES` variables usually have an empty placeholder in the application's Makefile so you can just add to them.*

Note: *The `SEARCH` variable is used other places in the build system. You must use `+=` when adding it to the Makefile so that your path doesn't overwrite other paths that have been included. For example:*

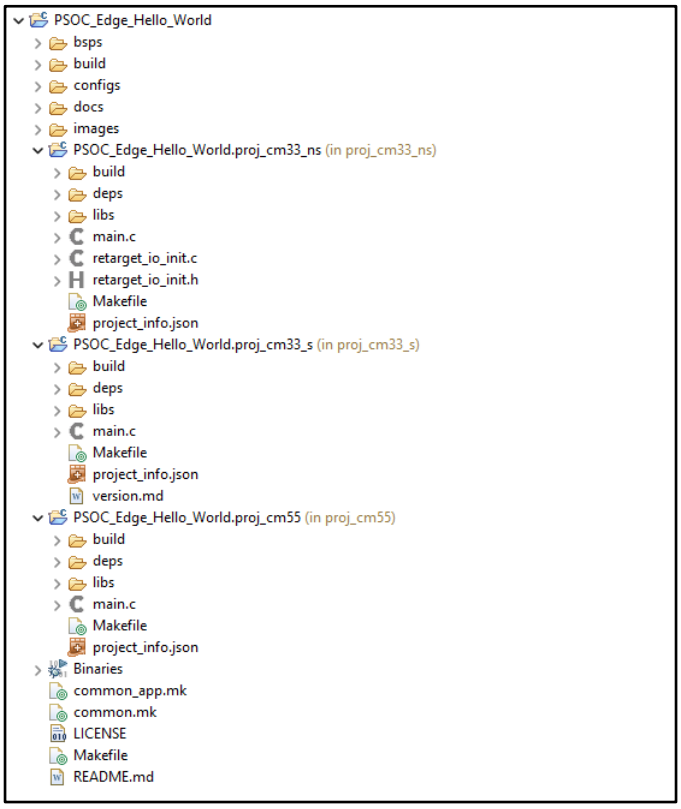
```
SEARCH += ../myLibraries
```

9.2 Multi-core applications

Multi-core applications have more than one project associated with a single application –one project per core. The top-level application directory contains application level files and a sub-directory for each of the projects. A dual-core application for an MCU with an Arm® Cortex®-M4 and an Arm® Cortex®-M0+ MCU may look like the following example:



Inside an IDE, the application will look similar to this:



Note: *There is a* deps *and* libs *directory inside each project, meaning each project has its own libraries. Since each core will be running separate code, each one has its own* main.c *file. Alternatively, since the entire application will be running on one physical board the* bsps *directory is at the application level.*

9.2.1 Makefile and make targets

For multi-core applications, there is a *Makefile* at the application level and one inside each project. There are also two files at the application level: one called *common_app.mk* that is included by the *Makefile* in the application and each project; *and one called common.mk that is included by each project's Makefile but not by the application's Makefile.*

Some operations can be done at both the application and project levels while other operations make sense only at the project level. A few of the common make targets and their behavior are shown here.

Make targets

Target	Level	Description
`help`	Both	`Print the list of make targets that are available at that level.`
`build`	Both	`Build all projects and generate a combined application hex image. The behavior is the same at the application and project levels.`
`build_proj`	Project	*Build only the selected project.*
`program`	Both	`Build all projects, generate a combined application hex image and program the combined image to the device. The behavior is the same at the application and project levels.`
`program_proj`	Project	*Build and program only the selected project to the associated core.*
`debug`	Project	Build only the selected project, launch the debugger for the target core, program the target core, start the application and stop in the target core's `main` function. *Note:* *From the command line, it is only possible to debug a single core at a time. However, some IDEs support simultaneous multi-core debugging.* *Note:* *You should program the other core before running the debug target since it only programs the core being debugged.*
`device-configurator`	Both	Run the Device Configurator. It will show the configuration for both cores and will allow you to specify which core each peripheral belongs to. `The behavior is the same at the application and project levels.`
`library-manager`	Both	Run the Library Manager. It will show libraries for each core and will allow you to manage libraries for each core separately. The BSP section applies to both cores. `The behavior is the same at the application and project levels.`

Make variables

Make variables used to control the build operate the same as variables for single-core applications. However, the make variables are split between the different files depending on how they are used (i.e. application only, project only, or both).

The `MTB_TYPE` variable is set to a different value than single-core applications. For the application level *Makefile*, it is set to `APPLICATION` while the variable for each project *Makefile*, is set to `PROJECT`.

One noticeable difference from single-core applications is that the top level *Makefile* has a variable called `MTB_PROJECTS` that lists the projects that are a part of the application. For example, the application shown above would include these lines in its *Makefile*:

Application level:
```
MTB_TYPE=APPLICATION
MTB_PROJECTS=proj_cm0p proj_cm4
```

The *common.mk* file (included in the CM0+ and CM4 projects) would have:

Project level:
```
MTB_TYPE=PROJECT
```

10 Library management

A ModusToolbox™ project is made up of your application files plus libraries. A library is a related set of code, either in the form of C-source or compiled into archive files. These libraries contain code which is used by your application to get things done. These libraries range from low-level drivers required to boot the chip, to the configuration of your BSP to Graphics or RTOS or CAPSENSE™, etc.

Whenever you add a new library to your application, the next step should be to look at the README.md file for the library. It can be found in the top-level library directory or in the library's GitHub repo. There is also a lonk to it in the Library Manager, which you will see later in this chapter. The README.md file will give you information on what is required to successfully use the library in your application. It will often contain a Quick Start section and a link to the full API for the library.

10.1 Library classification

The way libraries are used in an application can be classified in several ways (shared vs. local, direct vs. indirect, and fixed vs. dynamic). Each classification is explained below.

10.1.1 Shared vs. local

Source code for libraries can either be stored locally in the application directory structure, or they can be placed in a location that can be shared between all the applications in a workspace.

Most applications reference libraries as shared by default. If necessary, different applications can use different versions of the same shared library. Sharing resources reduces the number of files on your computer and speeds up subsequent application creation time.

You can easily switch a shared library to become local to a specific application, or back to being shared by using the Library Manager, which is discussed in section 10.3.

In earlier releases, a different library management method was in use. That method is not supported in ModusToolbox™ 3.x so it will not be discussed in this training.

The source code for all libraries is placed into a shared directory by default. The default directory name is *mtb_shared* and its default location is parallel to the application directories (i.e. in the same application root path). The name and location of the shared directory can be customized using the *Makefile* variables `CY_GETLIBS_SHARED_NAME` and `CY_GETLIBS_SHARED_PATH`.

The `CY_GETLIBS_SHARED_PATH` variable will sometimes be changed for applications with hierarchy such as multi-core applications. For most single-core applications, the path is set to *../* to locate the directory one level up from the application directory (i.e. parallel to it). In the case of multi-core applications, there is an additional level of hierarchy so the path is set to *../../* in each of the project *Makefiles*.

Source code for local libraries will be placed in the *libs* directory inside the application.

10.1.2 Direct vs. indirect

Libraries can either be referenced directly by your application (i.e. direct dependencies) or they can be pulled in as dependencies of other libraries (i.e. indirect dependencies). For example, some applications require the retarget-io library, so they include that library as a direct dependency. In other applications, there will be additional direct dependencies such as Wi-Fi libraries or Bluetooth® libraries. BSPs require various libraries to work with the device such as the HAL and PDL, so those libraries are included as indirect dependencies.

10.1.3 Fixed vs. dynamic versions

You can specify a fixed version of a library to use in your application (e.g. release-v1.1.0), or you can specify a major release version with the intent that you will get the latest compatible version (e.g. latest-v1.X). By default, when you create an application using the Project Creator tool, the libraries will be converted to fixed versions, so the versions of libraries in a given application will never change unless you specifically request it. This is called "latest locking" and it is done automatically during project creation based on information in the manifest file (more on manifests in section 15). The Library Manager can be used to control fixed vs. dynamic versions for each library.

Note: *If you don't use the Project Creator tool to create the project, latest locking is not done so it is important to use the Library Manager to select fixed version for each library unless you want them to dynamically change to the latest version whenever the Library Manager or* `make getlibs` *is run on the application.*

10.2 Including libraries and dependencies

There are two basic ways of including a library and its dependencies into your application: using a manifest file or using manual methods. There is also a third method that is only used for custom BSPs. Each method is discussed separately below. The method you should use depends on whether or not the library and its dependencies are specified in a manifest file.

10.2.1 Including libraries that are specified in a manifest

All libraries provided by Infineon are specified in manifest files that are automatically found by the tools. The manifest files specify where each library can be found and what each library's dependencies are. The manifest can also specify specific versions of libraries and dependent libraries that are meant to work together.

You can create your own manifest file(s) for your libraries so that they will show up in the Library Manager and will work just like the Infineon libraries. More information on manifest files and how to create and use your own manifests is covered in section 15.

In order to manage libraries that are specified in a manifest file in the application, files with the extension *mtb* are used. Their location and behavior depend on whether the library is a direct dependency or an indirect dependency. Normally the Library Manager is used to create/modify *mtb* files so you won't need to change them or even view them, but it is worthwhile to understand what is in them.

For direct dependencies, there will be one or more *mtb* files somewhere in your project (typically in the *deps* directory but the files could be anywhere except the *libs* directory). An *mtb* file is simply a text file with the extension *mtb* that has three fields separated by #:

- A URL to a Git repository somewhere that is accessible by your computer such as GitHub. This may use a special URL starting with *mtb://* as described below.
- A Git Commit Hash or Tag that tells which version of the library that you want
- A path to where the library should be stored in the shared location (i.e. the directory path underneath *mtb_shared*).

A typical *mtb* file looks like this:

```
https://github.com/cypresssemiconductorco/retarget-io#latest-
v1.X#$$ASSET_REPO$$/retarget-io/latest-v1.X
```

The variable `$$ASSET_REPO$$` points to the root of the shared location, specified in the application's *Makefile*. Use `$$LOCAL$$` in place of `$$ASSET_REPO$$` in the *mtb* file before downloading the libraries to make a library local to the app instead of shared. Typically, the version is excluded from the path for local libraries since there can only be one local version used in a given application. Using the above example, a library local to the app would normally be specified like this:

```
https://github.com/cypresssemiconductorco/TARGET_ KIT_PSC3M5_EVK#latest-v1.X#$$LOCAL$$/KIT_PSC3M5_EVK
```

Note: *The examples above specify dynamic library versions (e.g. latest-v1.X). Normally in your application, you will want fixed library versions (e.g. release-v1.0.0). This behavior can be controlled using the Library Manager.*

For standard Infineon libraries, an alternate format can be used for the URL starting with `mtb://` instead of `https:/github.com/cypresssemiconductor/` or `https://github.com/infineon/`. The tool knows where to look on GitHub for Infineon libraries, so the result is the same. For the shared library example shown above, the equivalent URL using the `mtb://` format is:

```
mtb://retarget-io#latest-v1.X#$$ASSET_REPO$$/retarget-io/latest-v1.X
```

Note: *When latest locking is performed on a library, the `mtb://` format in the mtb file is converted to the actual URL on GitHub (e.g. `https://github.com/infineon/`).*

For indirect dependencies, an *mtb* file for each library is automatically created in the *libs* directory. The version of each dependent library is also captured in the file *locking_commit.log* in the *deps* directory.

Once all the *mtb* files are in the application (both direct and indirect), the libraries they point to are pulled in from the specified Git repos and stored in the specified location. For example, *mtb_shared* is used for shared libraries and *libs* is used for local libraries. Lastly, a file called *mtb.mk* is created in the application's *libs* directory. The build system uses the *mtb.mk* file to locate all the libraries required by the application.

Since the libraries are all pulled in using `make getlibs`, you don't typically need to check them in to a revision control system. Instead they can be recreated at any time by re-running `make getlibs`. This includes both shared libraries (in *mtb_shared*) and local libraries (in *libs*). They both get pulled from GitHub when you run `make getlibs`.

The *mtb* files for indirect references and the *mtb.mk* file are stored in the *libs* directory and do not usually need to be checked in. In fact, the default *.gitignore* file found in our code examples excludes the entire *libs* directory. You should not need to check in any files from that directory and they can be recreated at any time.

10.2.2 Including libraries are not specified in a manifest

If you have your own custom libraries, you can create and include your own custom manifest files so that your libraries will show up in the Library Manager just like the Infineon libraries. This method also allows you to specify the dependencies in the manifest so that they are automatically added as indirect dependencies when the dependent library is added to the application. Custom manifests are covered in section 15.

If you do not want to create a manifest file for your custom libraries then you must manually include the library and its dependencies in each application that uses the library. Including the library can be done several different ways including:

- `git clone` the version of the library that you want (or use some other revision control system) into the application's directory

- Unzip an archive file of the library into the application's directory
- Recursive copy the library into the application's directory
- Modify the SEARCH variable in the application's *Makefile* to point to the library
- Add an *mtb* file to the *deps* directory for the library and run `make getlibs`, which requires that the library is in a Git repo

For the first four methods, the library can go anywhere in the application's directory tree. Remember that if you put it in the *libs* directory it will not be checked into version control by default, so you will normally want to put manually added libraries somewhere else.

If you choose to use the *mtb* file method, there are two things to be aware of: (1) the library will still not show up in the Library Manager; and (2) latest locking will not be performed on that library. You should use a fixed version in the *mtb* file unless you want the dynamic update behavior.

Adding dependencies can be easily achieved by using the Library Manager tool (for dependencies that are in a manifest file such as Infineon libraries). For dependencies that are not in a manifest file, you can use the same manual methods that are used for including the library itself.

10.2.3 Specifying dependencies for custom BSPs

Custom BSPs use an alternate method to specify dependencies. The advantage is that the specification of the dependencies is self-contained within the custom BSP itself.

This method is accomplished by having files with the extension *mtbx* for each dependency in the *deps* directory inside the custom BSP. An *mtbx* file has the exact same content as an *mtb* file with a different file extension.

The process of getting dependencies from *mtbx* files only searches in the target BSP for the application. Therefore, if your application contains multiple custom BSPs, only the dependencies for the active target (as specified in the `TARGET` variable in the *Makefile* or on the command line) will be included in the application.

The *mtbx* files are automatically created for you and you typically don't need to do anything with them unless you want to add or remove dependencies from your custom BSP. The BSP Assistant can be used to do this. It will be discussed in section 12.1.

During `make getlibs`, *mtbx* files are treated just like indirect dependencies that are specified in a manifest file. That is, the process will copy each *mtbx* file to the application's *libs* directory and will lock the version in the *locking_commit.log* file in the *deps* directory. The libraries are then pulled down according to the information contained in the *mtb* files.

10.3 Library Manager tool

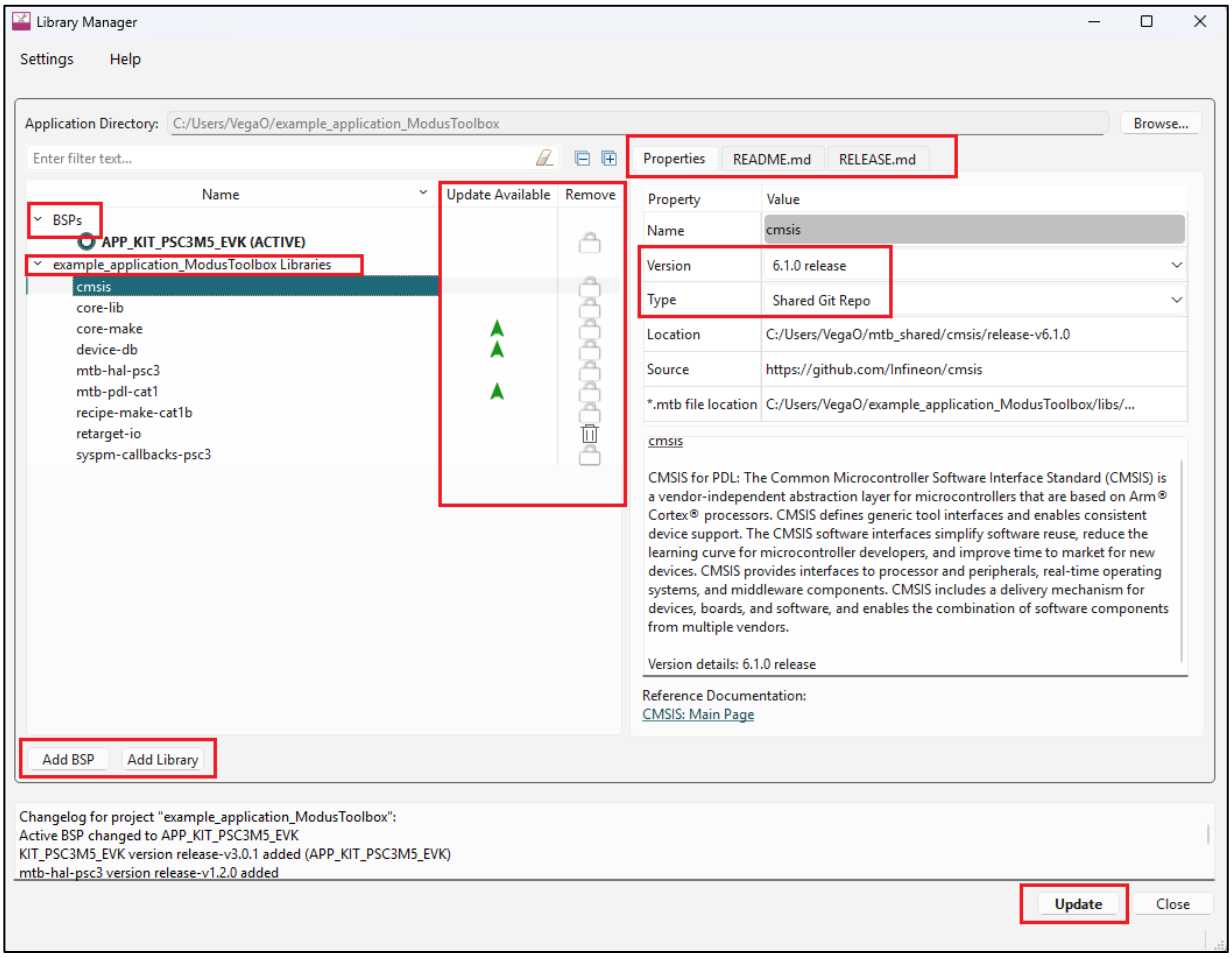
The ModusToolbox™ ecosystem provides a GUI for helping you manage library files. There are many ways to run it, such as:

- From the VS Code Application tab
- From the CLI, run `make library-manager` from the application directory
- Launch from the Windows **Start** menu
- Enter through `library-manager` in the Windows search box.
- Launch from the installation directory. For example, *ModusToolbox/tools_<version>/library-manager*

The Library Manager knows where to find libraries and their dependencies, using manifest files, which will be discussed in section 15. Therefore, *mtb* files included manually in your application that are not in a manifest file will *not* show up in the Library Manager.

Note: *The Library Manager never changes anything on disk until the **Update** button is clicked.*

Meanwhile, the Library Manager reads/creates/modifies *mtb* files, creates/modifies the *mtb.mk* file, and then runs `make getlibs`. When you change a library version and click **Update for example**, the Library Manager modifies the version specified in the corresponding *mtb* file and then runs `make getlibs` to get the specified version.



As you can see, there is a section for *BSPs* and a section for *Libraries*. If you are working with a multi-core application, there will be a section with libraries for each project.

The *BSPs* section allows you to specify which BSP should be used when building the application. An application can contain multiple BSPs, but a build from either the IDE or command line will build for and program to the "ACTIVE" BSP selected in the Library Manager.

The **trash can** symbol in the **Remove** column allows you to remove a BSP from the application, but you always need at least one BSP in an application.

In the Libraries section, clicking the **trash can** symbol in the **Remove** column removes the library. A lock symbol means that the library is a dependency of the BSP or another library and as such it cannot be removed.

The **Update Available** column shows when a library has a newer version available than the one being used in the application. A single green arrow indicates that there is a new minor version of the library available, while two green arrows indicates that there is a new major release of the library available.

In the right panel, information about the selected BSP or library is shown. There are tabs for **Properties**, **README.md** and **RELEASE.md** if applicable for he selected library.. The **Properties** tab is where you can make changes to versions, locations, etc. Some of the commonly used items are:

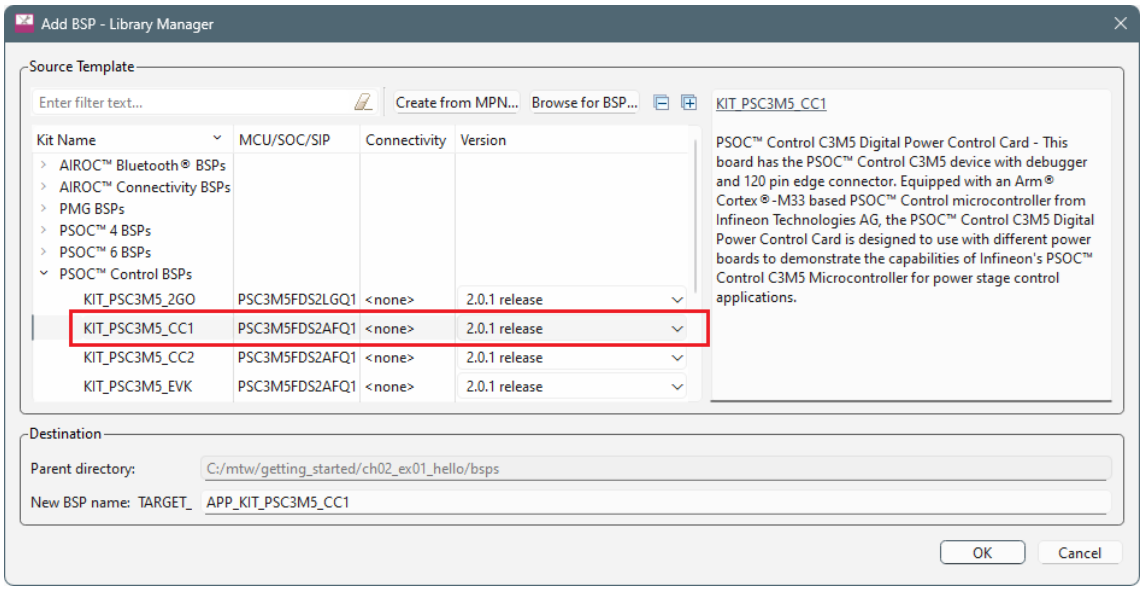
Version: You can select a specific fixed version of a library for your application or choose a dynamic tag that points to a major version but allows `make getlibs` to fetch the latest minor version (e.g. Latest 1.X). The drop-down will list all available versions of the library.

Type: Libraries can be a "Shared Git Repo" or "Local Git Repo." By default, most libraries are shared, meaning that the source code for the libraries is placed in the workspace's shared location (e.g. *mtb_shared*).

If you change the value (either version or type) for an indirect library, it will become a direct dependency. To change it back to an indirect dependency, just click the **trash can** next to the library name to remove the direct dependency.

Note: *The Version and Type cannot be changed for a ModusToolbox 3.X format BSP since it is flat data inside the application.*

The **Add BSP** button allows you to pull an additional BSP into the application. It opens a window like this:

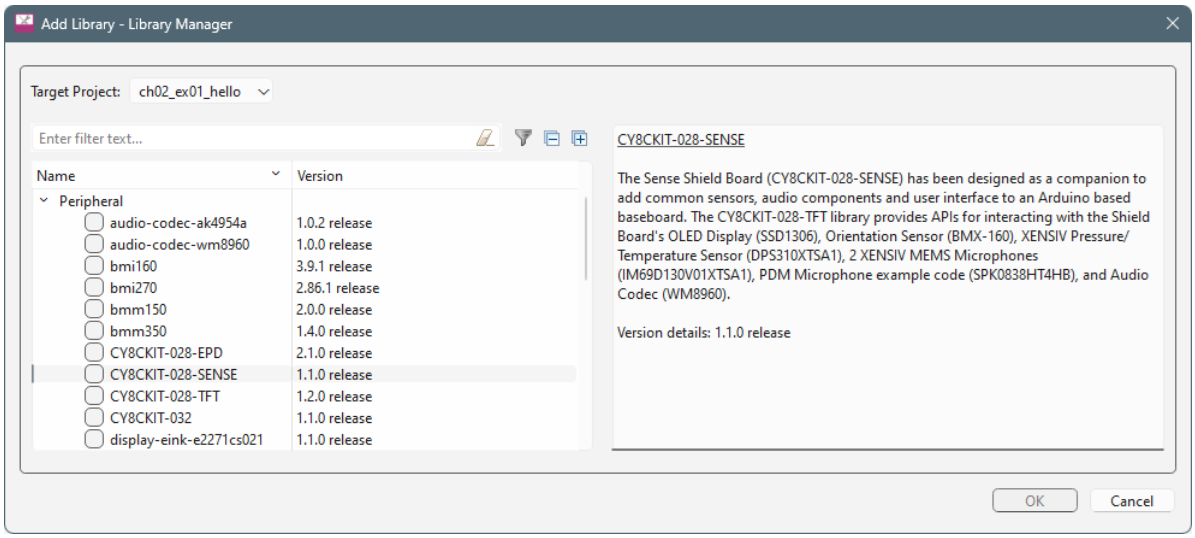


On this window, you have a few options to add a BSP:

- Select a standard BSP from the list. You can use the **+** and **-** buttons at the top to expand/collapse all categories and the text box to filter out specific BSPs.
- Click **Create from MPN** to launch the BSP Assistant and create one on the fly based on the device(s) you select.
- Click **Browse for BSP** to specify the path to a custom BSP.

Once selected, you can override the default name for the new BSP that will be created.

The **Add Library** button allows you to add new libraries. It opens a window like this:



You can use the **+** and **-** buttons at the top to expand/collapse all categories and the text box to filter out specific libraries.

Click the check box next to one or more libraries to add them to the project. You can select a different version for the library if you don't want the default. The filter button will show only libraries that are currently checked.

If you are working with a multi-core application, the drop-down menu at the top allows you to select which project you want to add the libraries to.

10.3.1 Active BSP

As described previously, one BSP is always selected as the active BSP. This is the target that will be used during a build. When the active BSP is changed, the Library Manager updates the `TARGET` variable in the application's *Makefile*.

When using VS Code, you must manually update the appropriate launch configurations after updating the active BSP or manually editing the `TARGET` in the *Makefile*. You can re-run `make vscode` to regenerate the launch configurations, but keep in mind that this may over-write other VS Code settings, so it is safer to update them manually. When it is re-run, the `make vscode` process will create a backup copy of the previous files in *.vscode/backup*.

10.4 Re-Downloading Libraries

The local library directory (*libs*) and shared library repo (*mtb_shared*) can be deleted at any time since they can be re-created using either `make getlibs` from the command line or using **Update** in the Library Manager. Typically, those directories should NOT be checked into a revision control system since they should only contain libraries that are already controlled and versioned. If you delete the mtb_shared directory from disk, you can easily re-acquire the libraries by using the Library Manager update function or by using make getlibs on an application.

11 Configurators

ModusToolbox™ software provides graphical applications called configurators that make it easier to configure hardware blocks and libraries. For example, instead of having to search through all the documentation to configure a serial communication block as a UART with a desired configuration, open the appropriate configurator to set the baud rate, parity, stop bits, etc. Upon saving the hardware configuration, the tool generates the C code to initialize the hardware with the desired configuration.

Note: *Normally during a build, the C code will be automatically regenerated if the configurator file is newer than the generated source files. Regeneration during a build can be suppressed by setting the make variable `SKIP_CODE_GEN` to any non-empty value. This is useful if you use source control for the generated source files and don't want them updated unless you explicitly decide to update them by re-running the configurator. If you use the `SKIP_CODE_GEN` variable and you use Git as your revision control system, you should remove GeneratedSource/ from the .gitignore file since it will prevent the GeneratedSource files from being checked in.*

Many configurators do not apply to all types of projects. So, the available configurators depend on the project/application you have selected in the Project Explorer. Configurators are included as part of the ModusToolbox™ installation. Each configurator provides a separate guide, available from the configurator's Help menu. Configurators perform tasks such as:

- Displaying a user interface for editing parameters
- Setting up connections such as pins and clocks for a peripheral
- Generating code to configure hardware block and libraries

Configurators are divided into two types:

- Board Support Package (BSP) Configurators
- Library Configurators

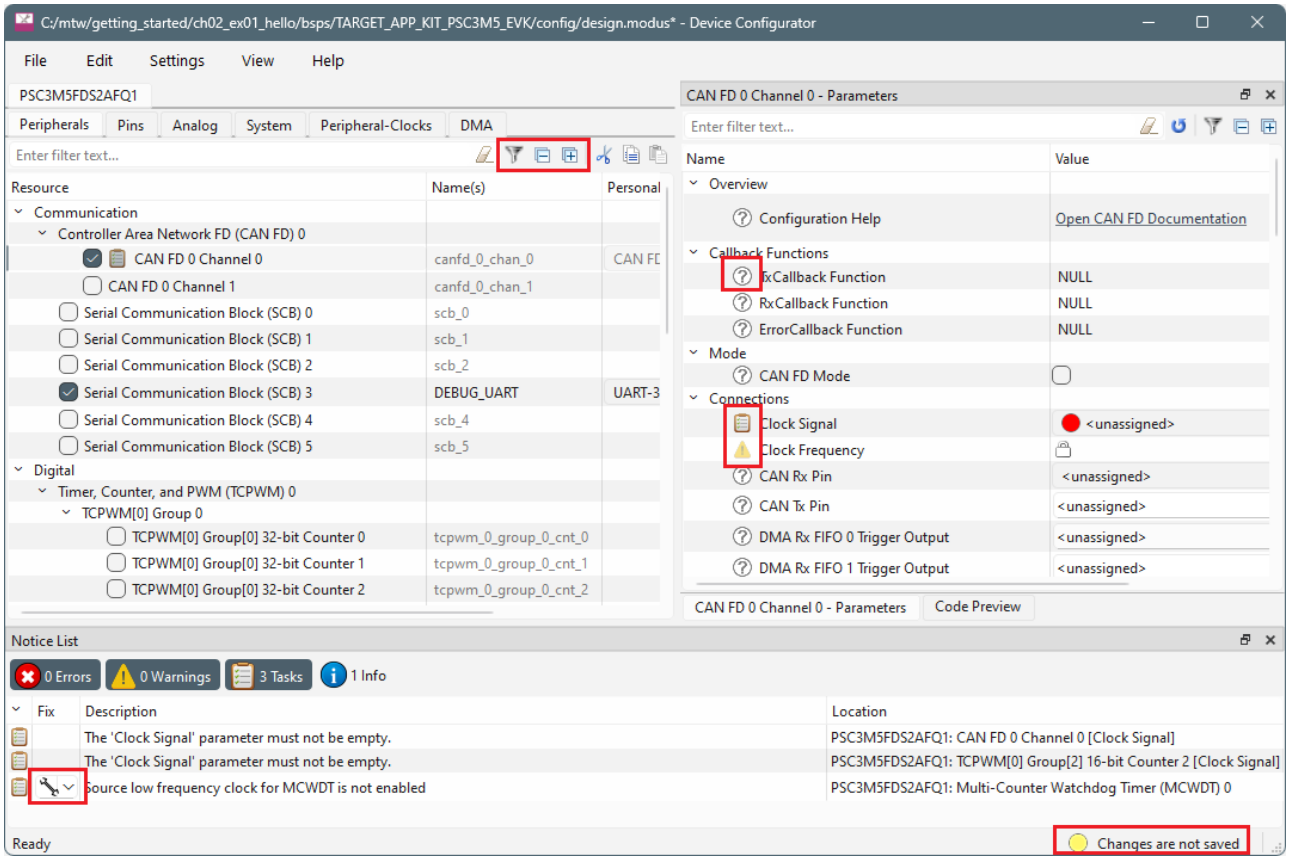
11.1 BSP configurators

BSP Configurators are closely related to the hardware resources on the board such as CAPSENSE™ sensors, external Flash memory, etc. As described earlier, these files are part of the BSP. These configurators can be launched from the BSP Assistant, which will be discussed in further detail later, or from within the application. The configurators are modifying files that are a part of the BSP regardless of which option you choose to launch with.

Note: *BSP configurators share the same file (design.modus). Therefore, you can only edit/save changes in one BSP configurator at a time. If you open a BSP configurator other than the Device Configurator and try to launch the Device Configurator, you will receive an error. If you launch another BSP configurator from inside the Device Configurator, the Device Configurator will be greyed out until the other configurator is closed.*

11.1.1 Device Configurator

Set up the system functions such as pins, interrupts, clocks, and DMA, as well as the basic peripherals, including UART, Timer, etc.



Note: *The **+** and **-** buttons expands/contracts all categories while the filter button shows only items that are selected. This is particularly useful on the **Pins** tab since the list of pins is sometimes very long.*

Note: *You can use copy/paste within the configurator to copy configurations for entire items. For example, you can copy/paste the entire configuration from one pin to another. To do this, just right-click on the item you want to copy from, select **Copy**, right-click on the item you want to copy to, then select **Paste**. If you use cut/paste, the old item will be disabled and the new item will be enabled.*

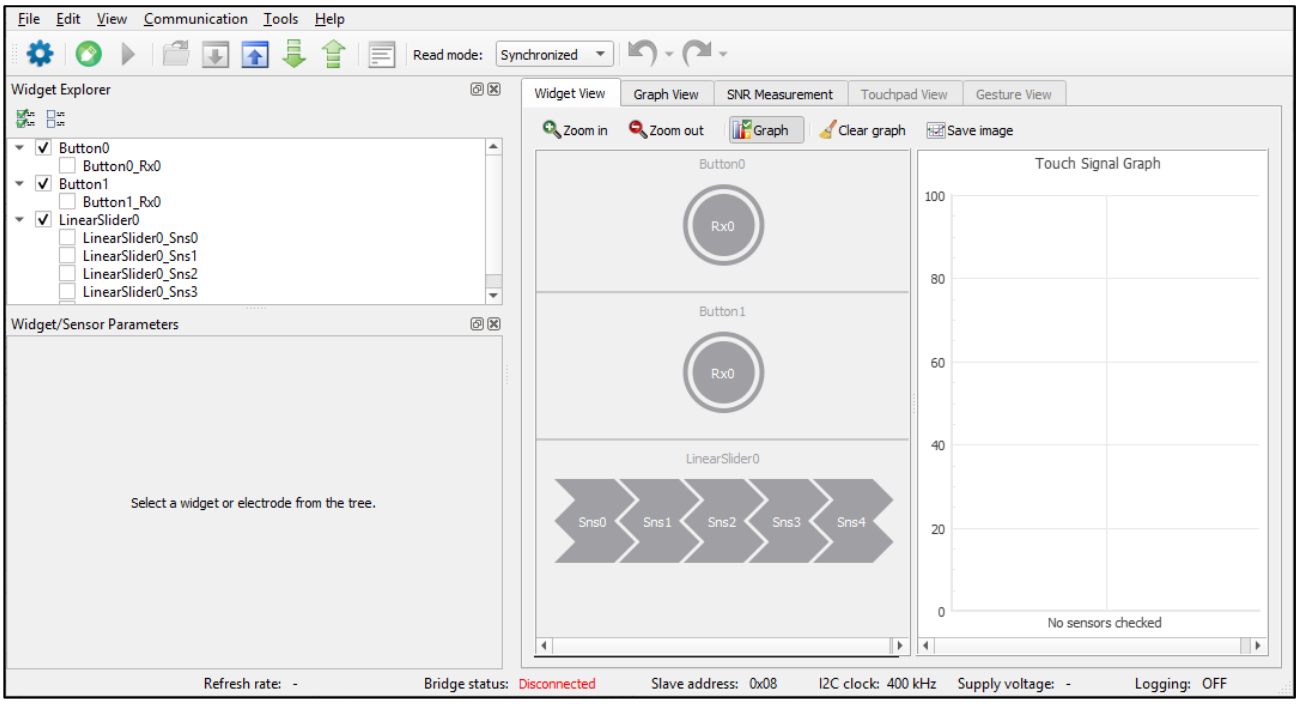
Several helpful icons will show up next to various items in the configurator. A few of interest are:

Icon	Name	Function	Example
	Chain	Clicking this icon will take you to the corresponding resource's configuration page.	When a peripheral has a pin associated with it, clicking the chain icon will take you to the configuration page for that pin.
	Notepad	This icon will appear when something needs to be configured.	If you enable a component that requires a clock, this icon will appear next to the clock selection row.
	Wrench	This icon appears in the log area when there are incompatible selections made across components. The drop-down will suggest fixes that can be applied automatically.	If you enable an ADC and select an internal reference, this icon will show up if the reference is not enabled. The drop down will allow you to enable the reference.
	Question Mark	This icon displays information about the parameter when you hover over it.	The Clock Frequency message may say "Source clock frequency."
	Exclamation Point	This icon displays an error message when you hover over it. Error icons will go away once items are properly configured.	Calculated parameters (for example, scan rate) cannot be determined if a clock has not been enabled for an ADC.

All other BSP configurators can be launched as either stand-alone or from inside the Device Configurator. To launch from inside the Device Configurator, enable the appropriate block and click the button in the **Parameters** pane. For example, for the CAPSENSE™ configurator, enable **Peripherals > CSD** and then click the **Launch CAPSENSE™ Configurator** button.

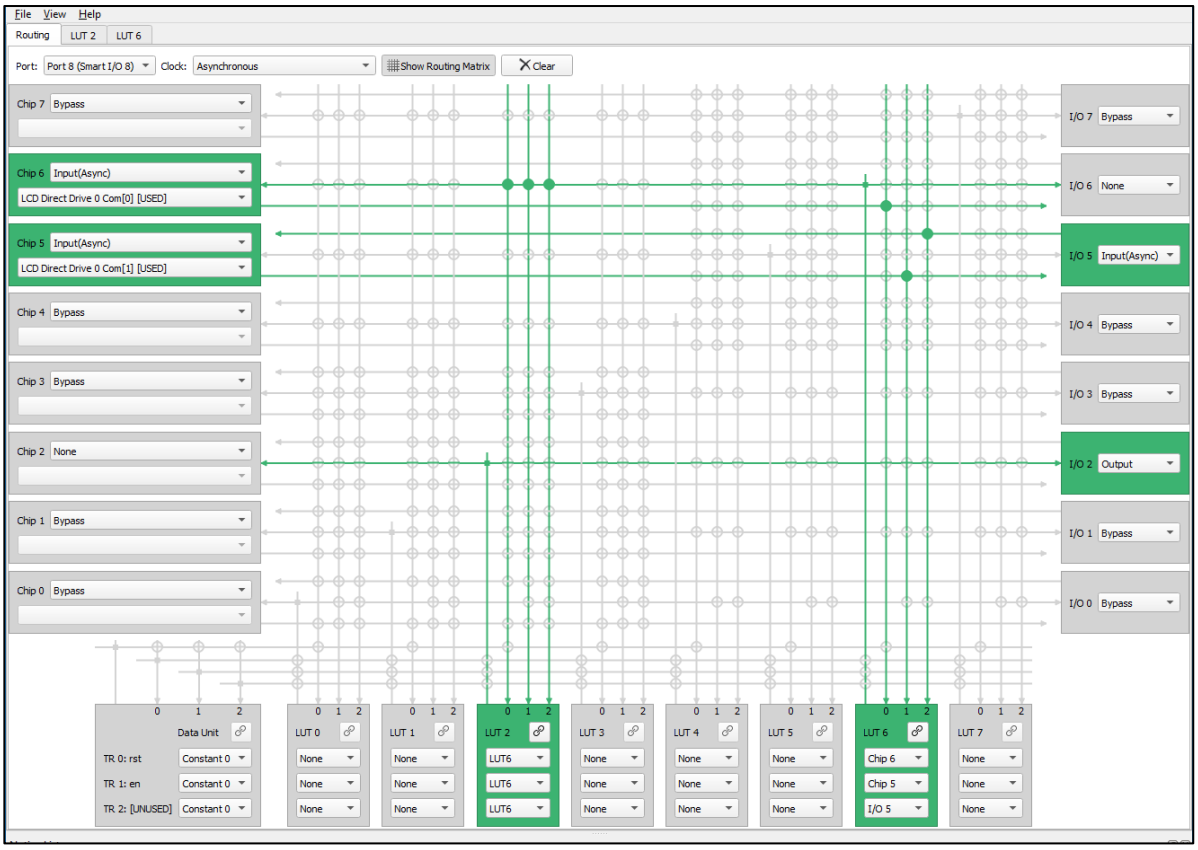
11.1.2 CAPSENSE™ configurator and tuner

The CAPSENSE™ block includes a configurator and tuner. The configurator allows the user to setup sensor configuration, pin mapping, scan parameters, etc. The tuner allows the user to view sensor data real time from the hardware to understand and improve its performance.



11.1.3 Smart I/O configurator

Configuring the Smart I/O block adds programmable logic to an I/O port. However not all MCUs have Smart I/O. For the MCUs that do, there are usually only a few ports with Smart I/O capability, which makes it important to consider that beforehand. For example, on the device from the KIT_PSC3M5_EVK, ports 0 – 3, 5 and 6, and 9 have Smart I/O. Each port operates independently - that is, signals from port 6 cannot interact with signals from port 9.



The port is chosen via a drop-down along the top of the window. Also selected at the top of the window is the clocking scheme. It can be asynchronous, or a variety of clock sources can be selected.

Along the left edge you will see a set of 8 chip connection points. These can be inputs to Smart I/O from chip peripheral outputs such as TCPWM signals or they can be outputs from Smart I/O to chip peripheral inputs such as SCB inputs.

The right edge allows connections to the chip's pins for that port - in our case, P8[7:0] and P9[7:0].

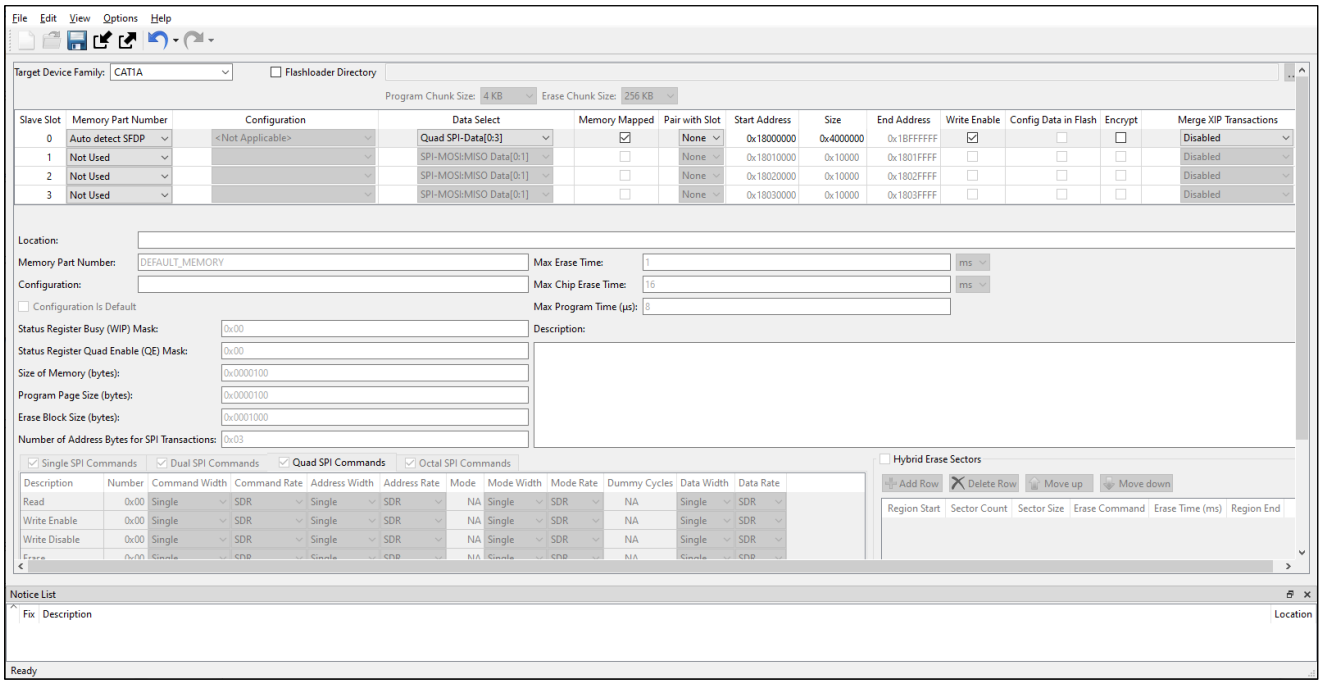
The bottom edge contains LUTs which can be used to combine and route signals to/from the chip's peripherals and to/from the chip's pins. The LUT inputs can come from peripherals, I/O pins, or from another LUT. Once you enable a LUT by selecting its inputs, a tab along the top appears for you to setup the truth table for the LUT. Note that if you don't need three inputs to a LUT, you can assign more than one input to the same signal.

The final section is the 8-bit Data Unit at the lower left corner. It allows you to setup trigger conditions that perform operations such as increment, decrement, and shift which are specified on the Data Unit tab once it is enabled by selecting one or more inputs.

Additional information is available in the Smart I/O user guide which can be found in the Help menu. You can also search for Smart I/O code examples in the usual places (In the Project Creator starter application list or on the Infineon GitHub site).

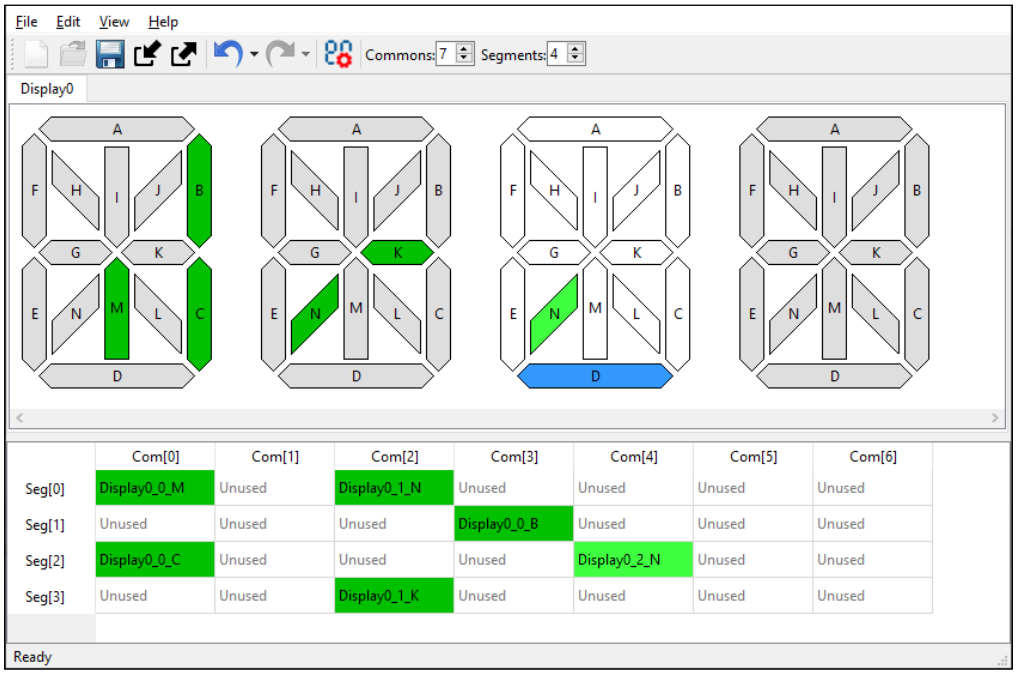
11.1.4 QSPI configurator

Configure external memory and generate the required firmware. This includes defining and configuring what external memories are being communicated with.



11.1.5 SegLCD configurator

Configure LCD displays. This configuration defines a matrix Seg LCD connection and allows you to setup the connections and easily write to the display.

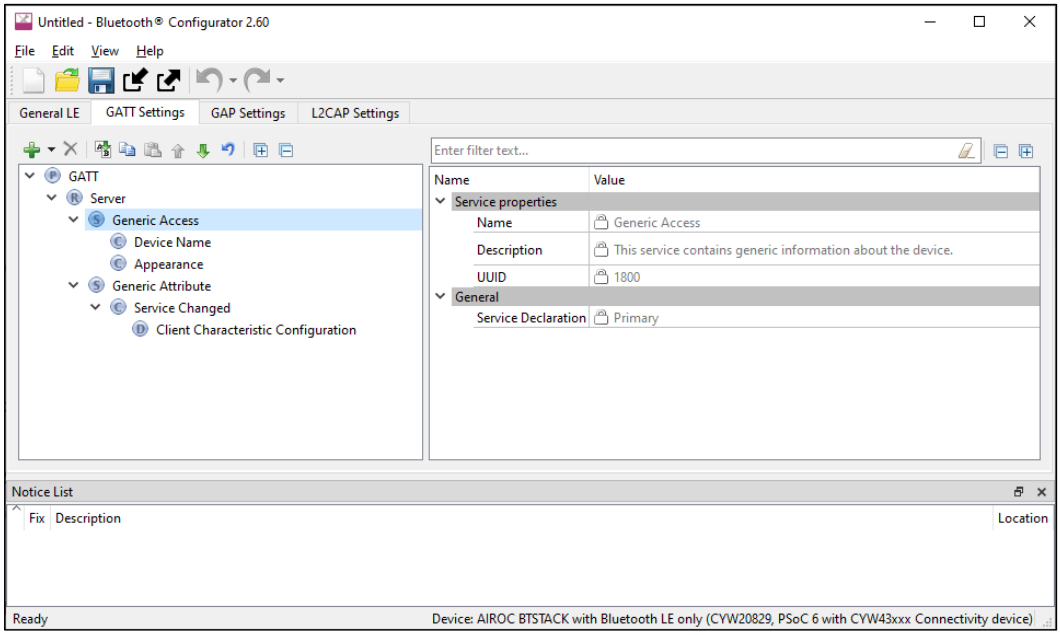


11.2 Library configurators

Library Configurators are used to setup and configure some software libraries. The files for these are contained in the project hierarchy rather than inside the BSP. Other configurators may be available depending on the device you are using. Each of the configurators has its own documentation accessible from the **Help** menu inside the tool.

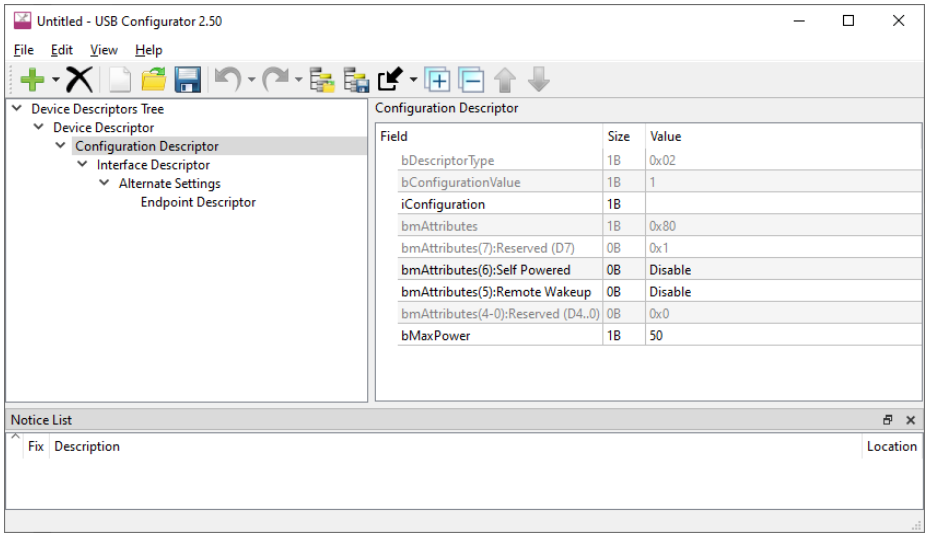
11.2.1 Bluetooth® configurator

Configure Bluetooth® settings. This includes options for specifying what services and profiles to use and what features to offer by creating GATT databases in generated code.



11.2.2 USB configurator

Configure USB settings and generate the required firmware. This includes options for defining the 'Device' Descriptor and Settings.



12 Working with custom BSPs

12.1 BSP Assistant

The ModusToolbox™ ecosystem provides a GUI for helping you create and edit BSPs.

12.1.1 Opening the tool

There are several ways to run the BSP Assistant:

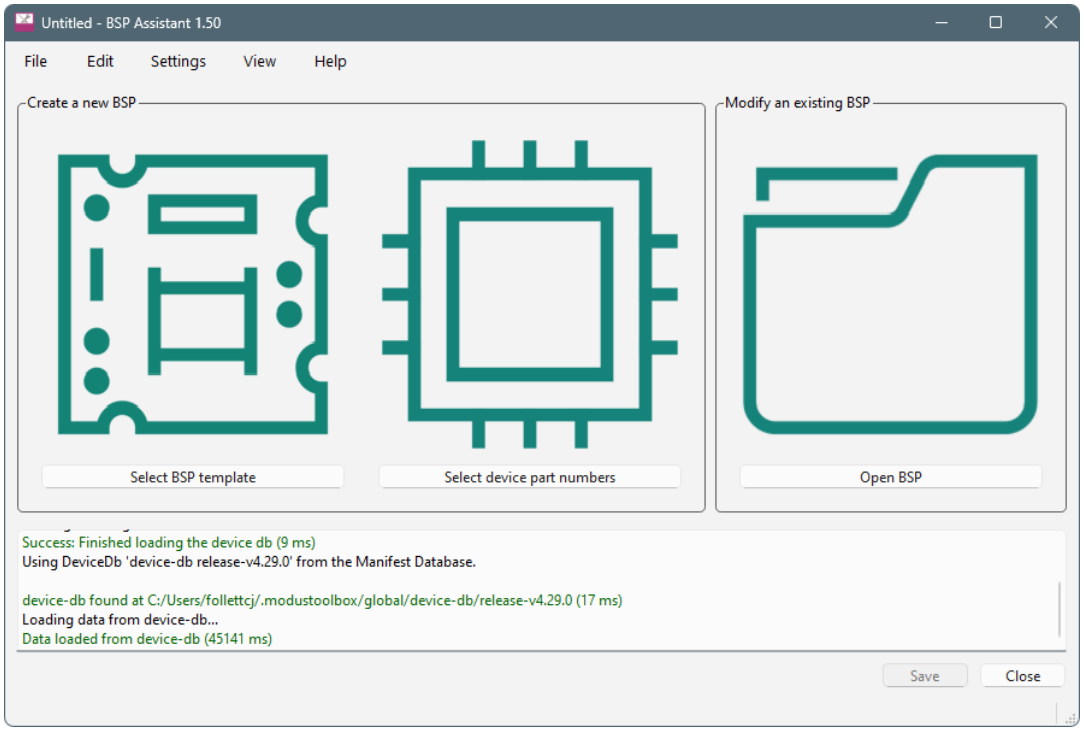
To open the active BSP from an existing application:

- From a make command terminal, run `make bsp-assistant`

To open the BSP Assistant GUI stand-alone:

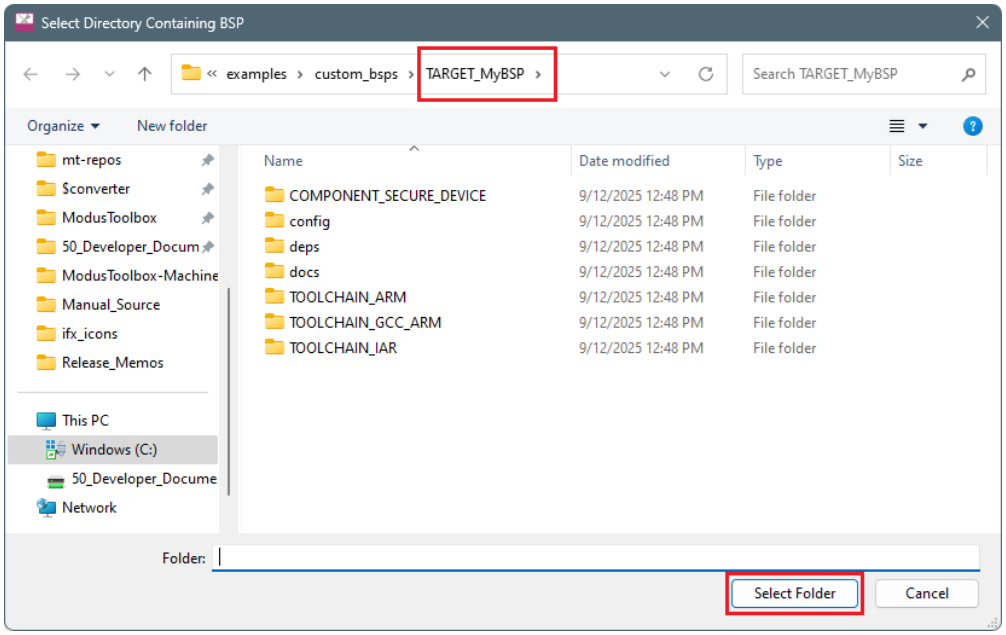
- Launch from the Dashboard
- Launch from the Windows **Start** menu
- Enter `bsp-assistant` in the Windows search box
- Launch from the installation directory. For example, *~/ModusToolbox/tools_<version>/bsp-assistant*

When you launch the GUI, it first loads the technology packs, tools information, and manifest data. Once that's done, if you opened the tool stand-alone, it will look like the following image. You can open an existing BSP, create a new BSP based on an existing one (either a standard Infineon BSP or a custom one) or create a new BSP by specifying the part numbers of the MCU and companion device(s) that are on the board. These actions can be done by clicking one of the links or using the **File** menu.



12.1.2 Open an existing BSP

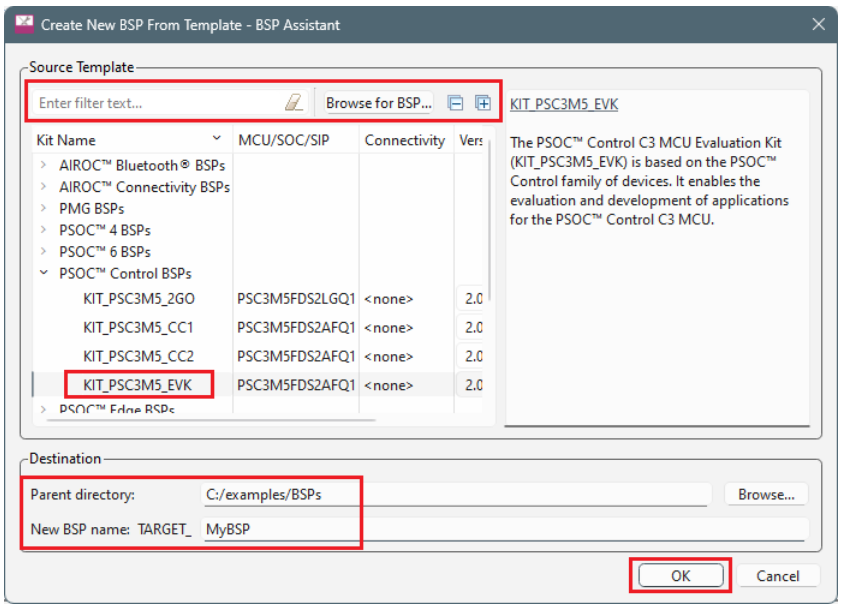
On the main BSP Assistant window, click the **Open BSP** link or select **File > Open** to open the standard Open dialog. Then, navigate to the directory that contains the desired BSP and click **Select Folder**.



Note: *If the specified BSP has a .git directory that points to an Infineon repository, a warning dialog will indicate that you might want to use the New... dialog to use the BSP as a template to create a new BSP in a new location that is not connected to Infineon's Git repositories before modifying it.*

12.1.3 Create a new BSP from an existing one

If you choose **Select BSP template**, a dialog opens that is similar to the BSP page in the Project Creator tool. Select a standard BSP to use as the template, or use the **Browse for BSP** button to specify an existing local BSP to use as a template.

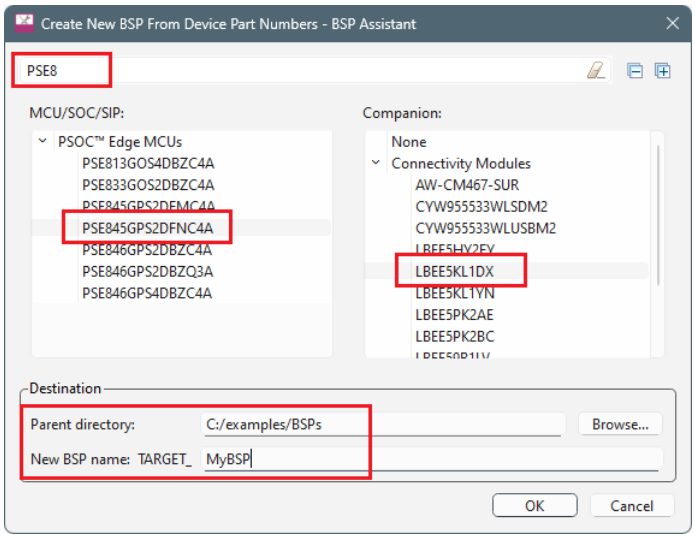


Use the filter box and buttons to expand/collapse all of the categories and make it easier to find what you are looking for.

Specify where the BSP will be saved and what it will be named and click **OK**.

12.1.4 Create a new BSP based on the device part numbers

If you choose **Select device part numbers** you will get a dialog like the one below.



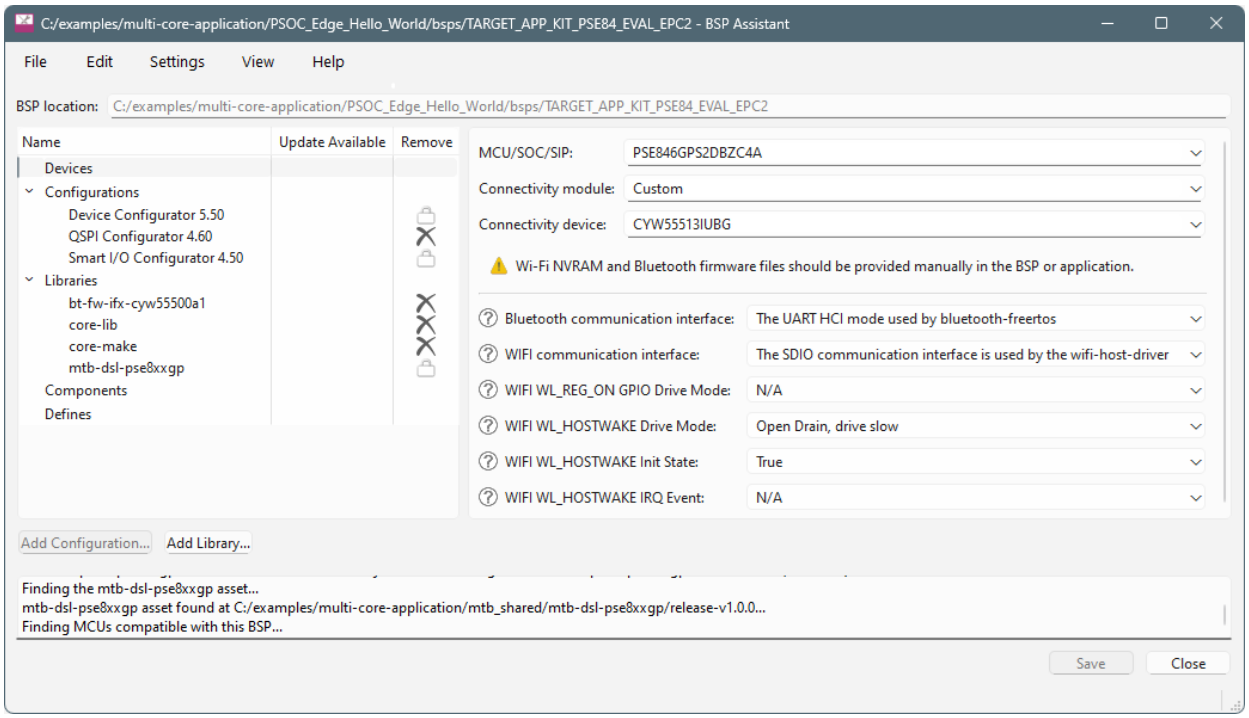
You must specify the MCU part number on your board and optionally the companion device such as a Bluetooth® or Wi-Fi module. Only companion devices that are compatible with the selected MCU will be listed.

Use the filter text box to narrow down the list of MCU devices by specifying all or part of the name.

Specify where the BSP will be saved and what it will be named and click **OK** to create the new BSP.

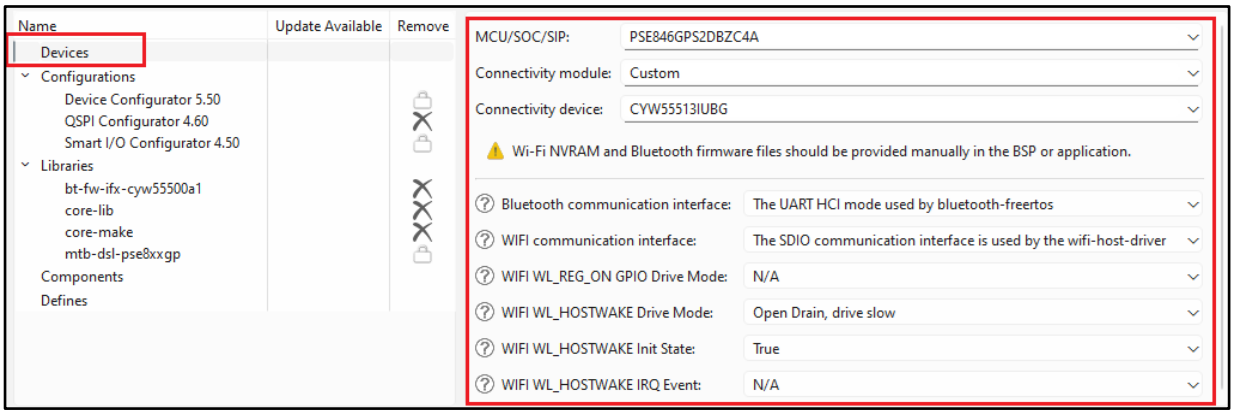
12.1.5 GUI description

Whether you create a new BSP or open an existing one, the main BSP Assistant window looks like the following image once the BSP is opened. There are five sections in the left panel: Devices, Configurations, Libraries, Components, and Defines.



Devices

If you click on Devices in the left panel, you can see and change selections for the device(s) that are on the board. You can also make selections based on the options available for the device(s) that are selected.



The MCU/SOC/SIP can be changed to any device within the same family. If you want a device in a different family, you must first create a BSP for a device in that family.

The connectivity module can be set to "None," any module that is supported for the selected MCU/SOC/SIP, or "Custom." In the case of "Custom," you must then specify the exact connectivity device that you are using and you must supply the required Bluetooth® and/or Wi-Fi firmware files for your device and hardware. If you choose a supported module, the appropriate files will be automatically provided.

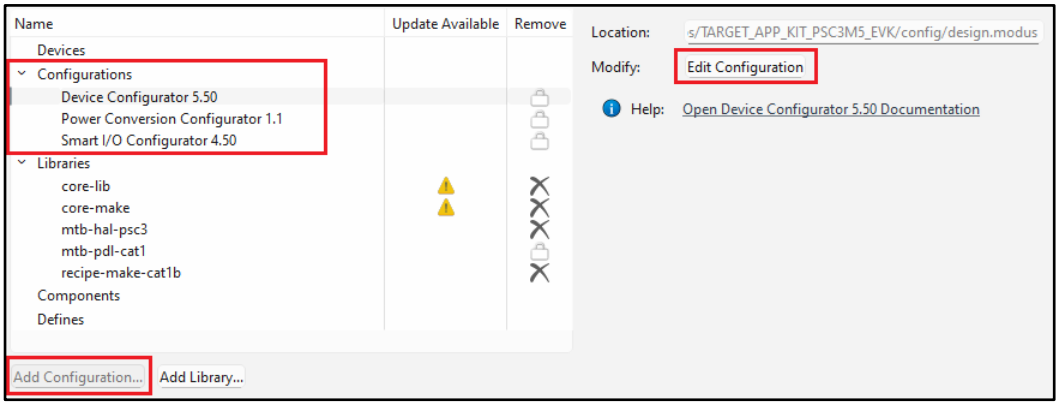
The remaining rows in the Devices section are options that are specific to the MCU/SOC/SIP and connectivity selections that you have made. These are used to select the settings required to get the desired behavior of the devices by setting `DEFINES` and `COMPONENTS` variables to the appropriate values in b*sp.mk*.

For example, "CM0+ pre-built firmware image" is used to select the firmware that is loaded onto a PSOC™ 6 MCU for the CM0+ core for a single-core application that only uses the CM4.

For a multi-core application, any pre-built firmware image must be disabled so that it can be replaced with the desired multi-core functionality. The method used to disable the pre-built firmware image is the `DISABLE_COMPONENTS` variable in the *Makefile*.

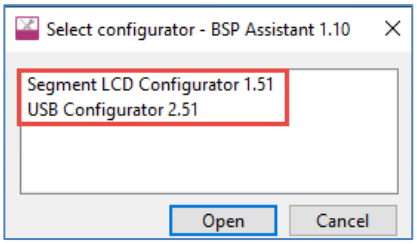
Configurations

The Configurators section allows you to edit, remove, or add BSP configuration settings for the selected MCU/SOC/SIP and connectivity device.



The list shows all configurators that currently have files in the BSP. You can click the **Edit Configuration** button to open the associated configurator to make changes or click the **X** button to remove a configurator file (some configurators are required and cannot be removed).

If the **Add Configuration…** button is enabled, you can get a list showing any available configurators not yet present for the selected device.

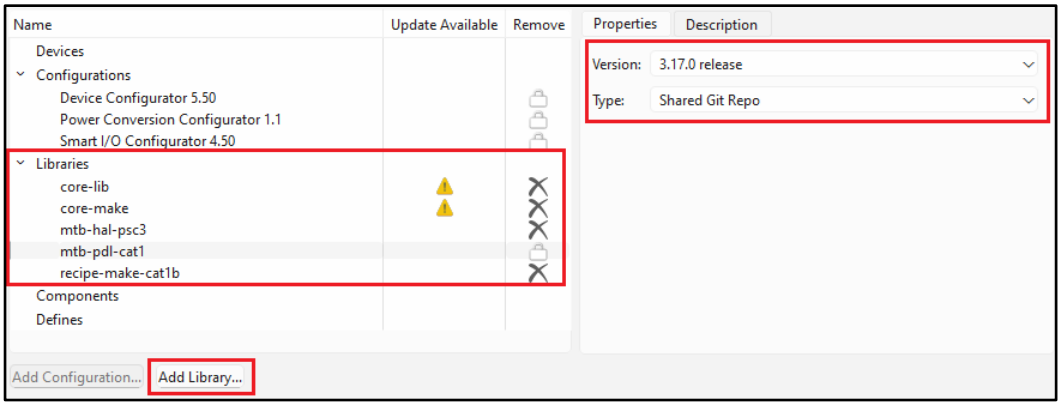


When you click **Open**, the associated configurator opens. Once you save changes from that configurator, it will appear in in the BSP Assistant list.

Note: *The BSP Assistant only operates on BSP configurator files. Library Configurators such as the Bluetooth® configurator store their files in the application rather than the BSP and they are not relevant to the BSP Assistant.*

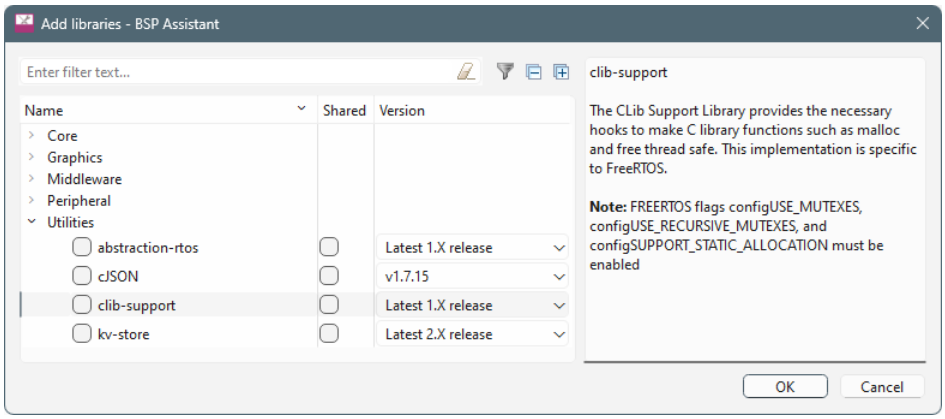
Libraries

The Libraries section shows all of the libraries that the BSP needs to operate along with any other libraries that have been added.



You can change the version of any dependency and the location where it will be placed for any applications that use the BSP. The location is either shared (i.e. in *mtb_shared*) or local (i.e. in the application's *libs* directory). You can remove libraries by clicking the **X** button.

Click the **Add Library** button to open a dialog allowing additional libraries to be added to the BSP.



From the dialog, check the box next to the library that you want to add, select the location (shared or not) where it will be placed in the application, and the version to use. Note that you can add as many libraries as you want at one time. Click **OK** once you are ready to add the library or libraries as BSP dependencies.

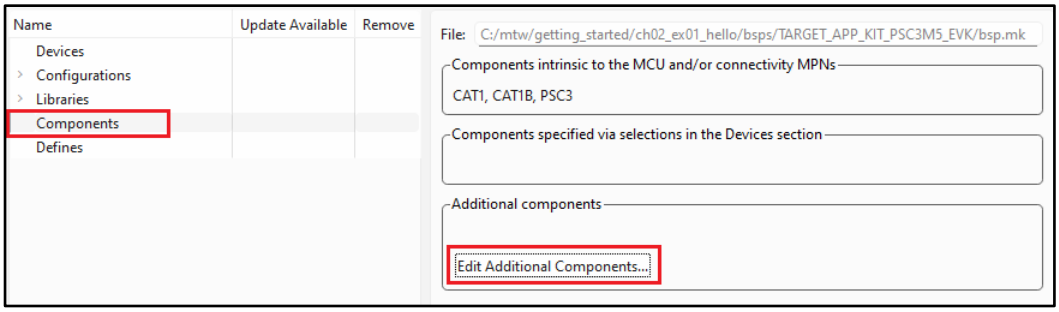
Note: *The default versions for the required libraries are the latest release versions at the time the BSP was created. If you want the libraries to be set to the latest versions when the BSP is used to create an application, you can change the versions to "Latest V.X release" where "V" is the major version number.*

Note: *When you add additional libraries, the default is "Latest V.X release" where "V" is the major version number. This means that those libraries will be set to the latest versions when the BSP is used to create an application. If you want fixed versions in the BSP, you can use the drop-down to select the release version that you require.*

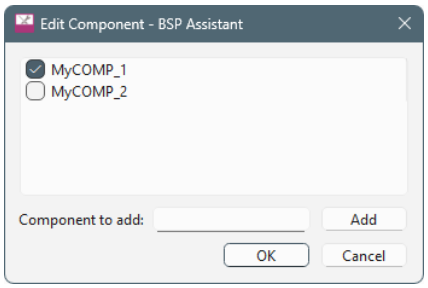
Note: *If you modify the libraries in a BSP, any application(s) that use the BSP must be updated to include those libraries. This can be done by running the Library Manager and clicking* ***Update*** *or by running* `make getlibs` *from the application's root directory on the command line.*

Components

The Components section lists values that are included in the `COMPONENTS` variable in the *bsp.mk* file.



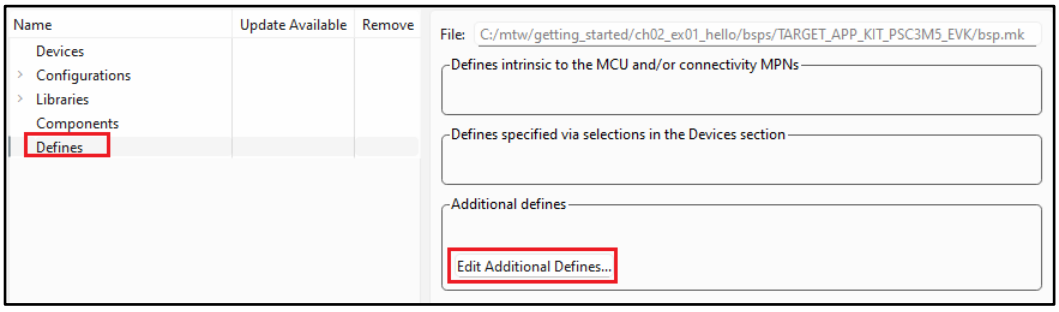
There are three sections: components that are required by the devices that your BSP contains, components that are required based on option selections from the Devices section, and any additional component values. The first two sets are set based on the Devices section so they cannot be modified in the Components section. Additional components can be added or removed by clicking the **Edit Additional Components** button.



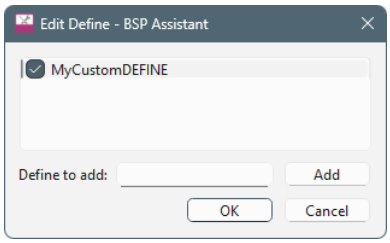
Type the name of a component to add in the box and click Add to add it to the list. Uncheck the box next to any existing components to remove them. Click OK when you are done making changes.

Defines

The Defines section is similar to the Components section except that it lists values for the `DEFINES` variable in the *bsp.mk* file.



As with the Components section, there are three categories of defines. The first two are read-only values that are based on the selections in the Devices section while the third is additional defines that can be modified. Click **Edit Additional Defines** to add or remove additional define values.



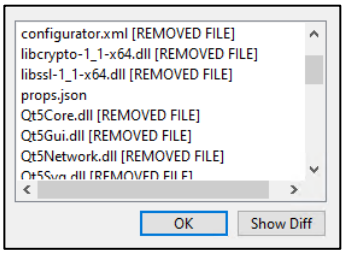
12.1.6 Save, Undo and Redo

Whenever you make changes, the **Save** button will become active. Changes are made to the BSP on disk only when you click **Save**.

Note: *Since the configurations each have their own independent configurator tool, saving from a configurator immediately changes the BSP files on disk. In addition, you must save all other changes before you can add or edit configuration files since changes in other areas may affect the configuration files.*

The **Edit** menu has **Undo** (Ctrl+Z) and **Redo** (Ctrl+Y) commands that allow you to undo and redo changes respectively until the BSP is closed.

The **View** menu has a command to **List Changed Files**. This will show a list of all files that have been changed since the BSP was opened. It also allows you to view differences between the old and new version of any changed file. Just double-click on a file or select it and click **Show Diff** to see the differences.



12.1.7 Updating launch settings

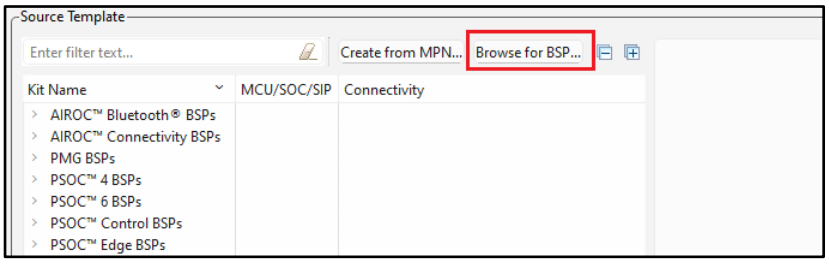
If you are using an IDE, you may need to regenerate the launch settings to reflect the new BSP. Use the appropriate command(s) for the IDE(s) that are being used. For example: `make vscode` for VS Code. You must complete this step whenever the target device is changed since the launch settings may change from device to device.

12.2 Using a custom BSP in a new application

When using a custom BSP that you created in a new application, you have a few options. Each option is discussed below.

12.2.1 Using a local BSP

The easiest way to include a custom BSP in a new application is to simply use the **Browse for BSP** button when selecting the BSP for the application in the Project Creator tool. Just browse to the directory containing the BSP and select it. The BSP will be copied into the application's *bsps* directory when the application is created.



12.2.2 Using a custom manifest for the BSP

As stated above, the browse mechanism is the simplest way to include a custom BSP in an application, but you can also get your custom BSP to appear in the list of kit names in the Project Creator and Library Manager tools. For this method to work, the custom BSP must be in a Git repo. You must then create a custom super-manifest file and a manifest file so that the Project Creator and Library Manager tools will know how to find the Git repo. The manifest file includes not only the location of the BSP's Git repo, but also the list of dependencies for the BSP. See section 15 for additional details.

12.2.3 Using a version-controlled copy of the BSP

If your BSP is in a version control system (e.g. Git, Subversion, etc.), you can place a controlled copy of the BSP inside the application. You must first create the application with a local BSP by browsing to the custom BSP as described above. Once the application is created, you can delete the local copy of the BSP from the application. Then use Git clone or any other mechanism supported by your version control system to create a controlled version of the BSP in the application.

Note: *Remember that if the BSP is a version-controlled copy, any edits (such as changes using the Device Configurator) will affect that version-controlled copy.*

13 Creating applications for other IDEs

For more details on working with the IDE of your choosing other than VS Code, refer to the following user guides:

- Eclipse for ModusToolbox™ user guide
- IAR Embedded Workbench for ModusToolbox user guide
- Keil µVision for ModusToolbox™ user guide

14 Version control and sharing applications

If you are working on a design with more than one person, it is common to want to share an application using some type of version control system such as Git. Even if you are not using version control, you may want to share applications manually copying files or using your IDE's export/import mechanisms.

14.1 Files to include/exclude

No matter which method you choose, you will want to know what is critical to check in to version control or copy, and what can be regenerated easily. The main files to consider when sharing or copying include anything that you have changed or added, and that will not be regenerated. These files include source code, BSPs and configurations, *Makefile*, etc. There are several directories in the application that can be re-created and therefore do not need to be copied or checked into version control. These include the *libs, mtb_shared*, *build*, and *GeneratedSource* directories. The processes that create them are:

libs and mtb_shared: Created by running `make getlibs` or by running the Library Manger and clicking the **Update** button. Either option will clone the libraries from GitHub to the appropriate locations.

build: Created during the build process.

GeneratedSource: Generated by running the associated configurator such as the Device Configurator or Bluetooth® configurator. The configurators are run automatically by the build process if the *GeneratedSoruce* files are out of date.

14.2 Using version control software

If you are working on a production design, you probably use version control software to manage the design and any potential revisions. This allows all users to stay synchronized with the latest version of an application. ModusToolbox™ assets are provided using Git, but you can use any version control method or software that you prefer.

ModusToolbox™ code examples have a default *.gitignore* file that excludes the directories that can be easily recreated and files containing IDE-specific information that need not be checked in. If you are using Git as your version control software, that file can often be used as-is, but you are free to change it if you want to change what gets checked in. For example, you may decide that you want to check in all of the libraries from *libs* and *mtb_shared* even though they are available on Infineon's GitHub site or you may want to check in the *GeneratedSource* directories even though they can be re-created from the configurator files during a build.

If you are using different version control software, the *.gitignore* file can be used as a guide for configuring the software that you are using.

Once you have an application checked into your desired version control software, sharing the application with a new user is straight-forward. The steps are:

1. Get a copy of the checked-in data. This will vary depending on the version control software (for example, `git clone <url>`).
2. Run the Library Manager and click the **Update** button, or open a terminal and run the command `make getlibs`. Either one will get all of the libraries required by the application.
3. Use the application as usual. The *build* and *GeneratedSource* files will be created automatically when needed.
4. Finish with your changes. Then check in your updates after completing your version control process.

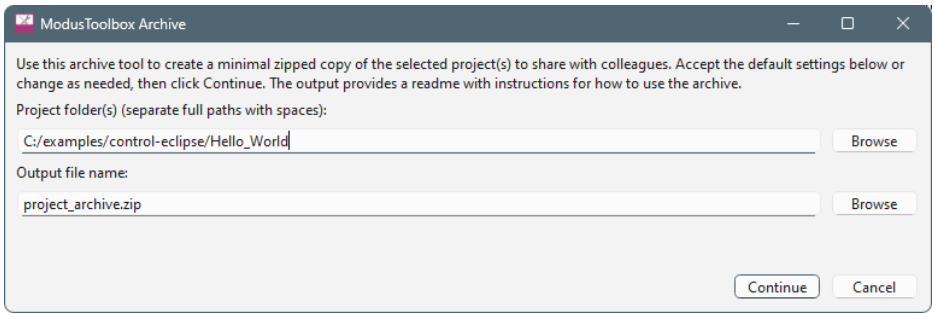
14.3 Manual file copy

If you are not using version control software, you can copy a complete application directory from one user to another. You can also choose to exclude the directories listed in 14.1. since the libraries can be downloaded by running `make getlibs` or running the Library Manager and clicking **Update**. The other files will regenerate once the application is built.

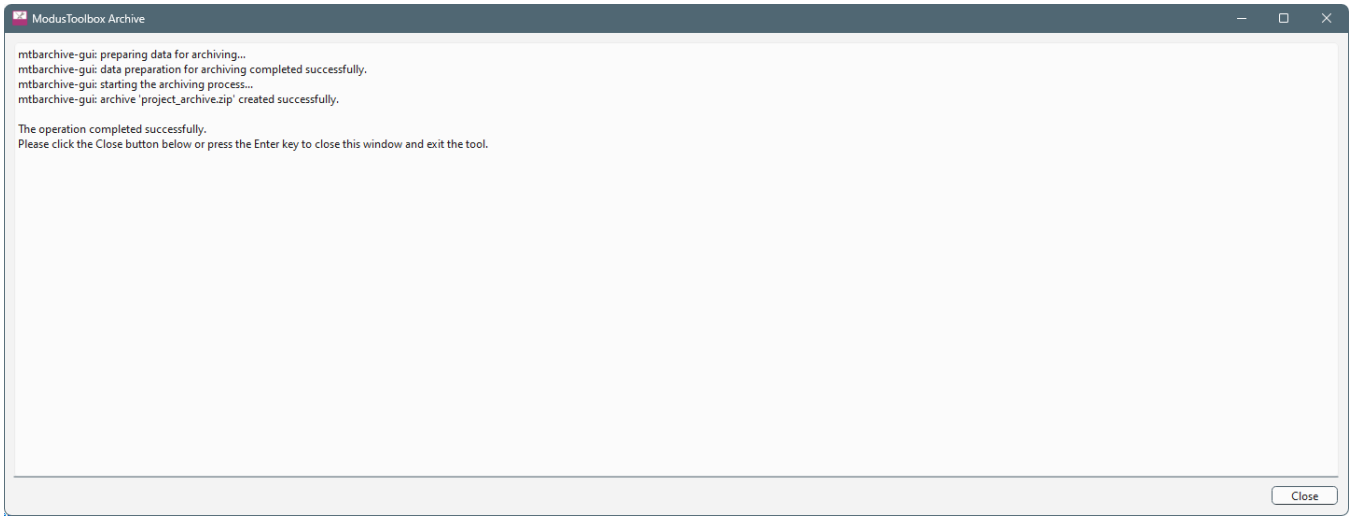
14.4 Using the Archive tool

The Archive tool (command line and GUI), provides a quick way to create a minimum sized project/application that you can share with a colleague. You can open the tool from VS Code, or just from the installation folder.

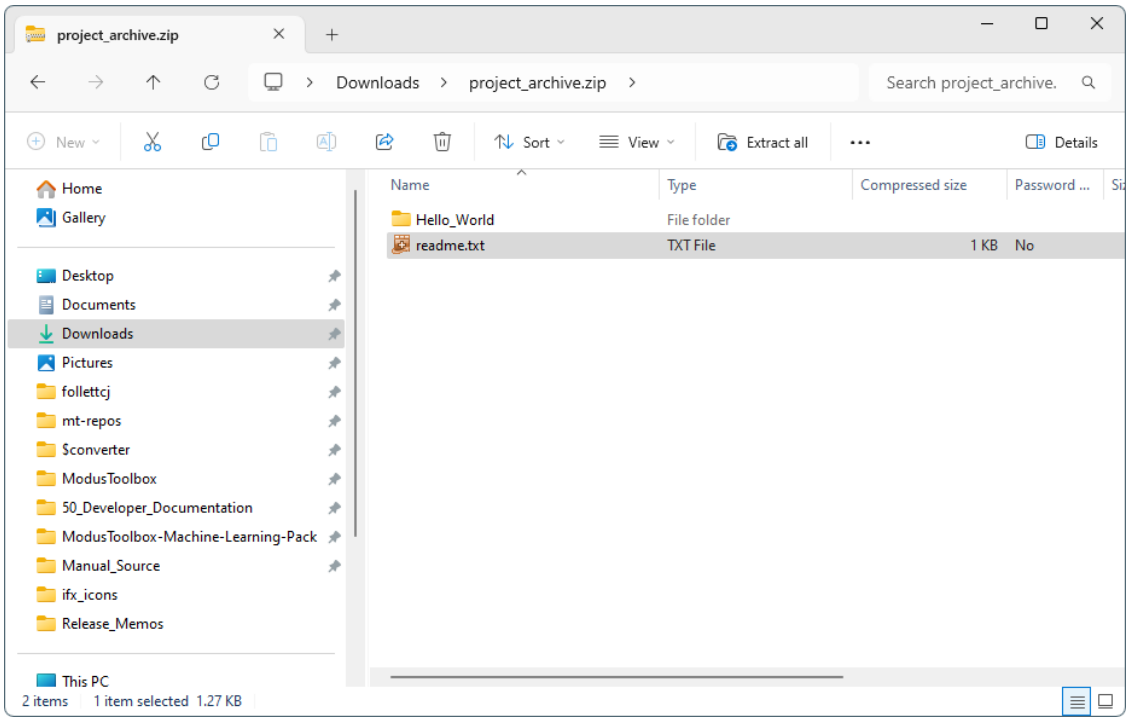
When opened from VS Code, the GUI tool selects the project to be archived by default, and provides a given archive name. You can add more projects separated by commas, and you can change the name of the archive.



When you click **Continue**, the tool prepares the archive ZIP file and removes all files that can be regenerated by the new user. After a few moments, the tool displays messages about the creation of the archive. It will also display any errors if they were encountered.



When you open the archive, you will see a readme.txt file included next to the project / application folder(s).



This file provides instructions to the new user for how to extract the archive and regenerate the files that were removed. It also includes information such as what projects/applications are included, and any packs that were used to create them.

15 Manifests

Manifests are XML files that tell the Project Creator and Library Manager how to discover the list of available boards, example projects, libraries and library dependencies. There are several manifest files.

- The "super manifest" file contains a list of URLs that software uses to find the board, code example, and middleware manifest files.
- The "app-manifest" file contains a list of all code examples that should be made available to the user.
- The "board-manifest" file contains a list of the boards that should be presented to the user in the new project creation tool as well as the list of BSP packages that are presented in the Library Manager tool. There is also a separate BSP dependencies manifest that lists the dependent libraries associated with each BSP.
- The "middleware-manifest" file contains a list of the available middleware (libraries). Each middleware item can have one or more versions of that middleware available. There is also a separate middleware dependencies manifest that lists the dependent libraries associated with each middleware library.

In order to use your own examples, BSPs, and middleware, you need to create manifest files for your content and a super-manifest, pointing to your manifest files.

Once you have the files created in an accessible location, place a *manifest.loc* file in your *<user_home>/.ModusToolbox* directory specifying the location of your custom super-manifest file, which in turn points to your custom manifest files. For example, a *manifest.loc* file may have:

```
# This points to the IOT Expert template set

https://github.com/iotexpert/mtb2-iotexpert-manifests/raw/master/iotexpert-super-manifest.xml
```

Note: *The super-manifest and manifest files must be accessible via a URL. However, they do NOT need to be stored on a Git server such as GitHub – they can use any URL that points to the xml files. For example:*

```
https://myserver.com/my-super-manifest.xml
```

Note: *You can point to local super-manifest and manifest files by using file:/// with the path instead of https://. For example:*

```
file:///C:/MyManifests/my-super-manifest.xml
```

To see syntax examples of super-manifest and manifest files, you can look at the Infineon manifest files on GitHub. The following is a subset of the full set of manifest repos that exist. Each repo may also contain multiple manifest files.

- Super Manifest: https://github.com/infineon/mtb-super-manifest
- Code Example Manifest: https://github.com/infineon/mtb-ce-manifest
- BSP Manifest: https://github.com/infineon/mtb-bsp-manifest
- Middleware Manifest: https://github.com/infineon/mtb-mw-manifest
- Wi-Fi Middleware Manifest: https://github.com/infineon/mtb-wifi-mw-manifest

16 Troubleshooting

The following sections discuss some common build and programming errors and how to resolve them.

16.1 Build support not found

An error message that says unable to find build support for this device means that the build system can't find a critical part of itself. The main pieces are:

Item	Location	Description
core-make	mtb_shared directory	The shared/central part of the build system.
recipe-make-cat<n>	mtb_shared directory	The recipe that tells core-make how to handle your selected device family.
TARGET_<bsp-name>	Application's *bsps* directory	The board support package. This tells the build system which exact devices are present and which recipe to use.

If you get this error, the first thing to do is to make sure you've run/rerun the command `make getlibs` either from the command line or by running the Library Manager and clicking the **Update** button. This is especially true if you've changed the TARGET variable in the Makefile to use a different device or if you have modified devices inside the BSP.

16.1.1 Path too long

Windows has hard limits on file path lengths. There are actually two limits:

> File Path: 260 characters including driver letter, colon, slashes, directory name, file name, file extension and terminating NULL character.
>
> Directory Path: 248 characters including everything above except the file name and file extension.

The exact error message you see will depend on the tool that tries to access the long file – make, compiler, configurator, openocd, etc. Often, you will see some sort of "file not found" error message.

If you have a long path error, you can use shorter names for the BSP and/or the application or you can move the application closer to the root of your drive. Alternately, you can use the Windows `subst` command to substitute a drive letter for part of the path. For example, if I have an application in *C:/Users/testuser/mtw/workspace1/project1*, I could do the following:

1. Open a Windows command terminal (i.e. `cmd`)
2. Enter the command `subst Z: C:\Users\testuser\mtw`
3. Run `modus-shell` and use the command `cd Z:` to get to the *mtw* directory
4. The build will now see the path to the application as *Z:\workspace1\project1*.

16.2 Incorrect device

The build system defers to OpenOCD to handle programming. There are no baked-in smarts or ability to tell which device is plugged in. For devices from different families, OpenOCD will generate errors, but in other cases you must review the log to see messages about the device being programmed versus what you expect.

17 Network considerations

The Project Creator, Library Manager and BSP Assistant tools require internet access in order to find assets such as BSPs, code examples, and libraries. Depending on your network and location, you may need to change proxy settings or work around other network access limitations.

17.1 Network proxy settings

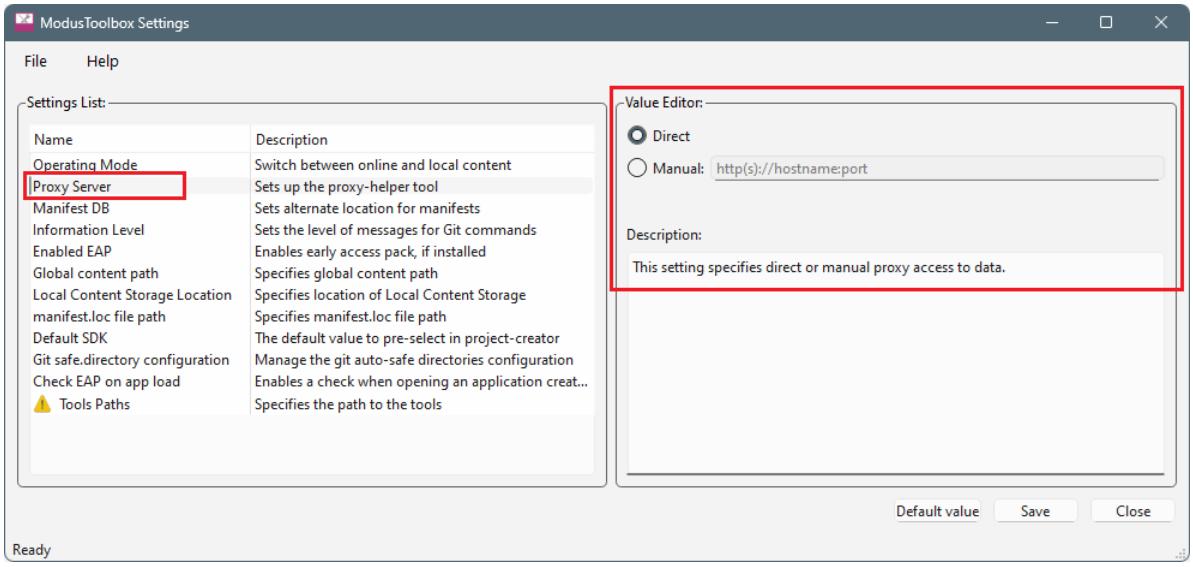
If you have environment variables for *http_proxy* and *https_proxy*, those same variables will work for ModusToolbox™ tools. However, there is a simpler way to enable/disable the proxy settings in the Project Creator and Library Manager tools that doesn't require you to create/delete the environment variables. This method also allows you to switch between a network that requires a proxy and a second network that does not require a proxy.

When you run the Project Creator or Library Manager, it first looks for a remote manifest file. If that file isn't found, you will not be able to move forward. In some cases, it may find the manifest but then fail during the project creation step (during `git clone`). If either of those errors occur, it is likely due to one of these reasons:

- You are not connected to the internet. In this case, you can choose to use offline content if you have previously downloaded it. Offline content is discussed in the next section.
- You may have incorrect proxy settings or no proxy settings.

To view/set proxy settings in the Project Creator or Library Manager, use the menu option **Settings > ModusToolbox Settings...** Then choose **Proxy Server**.

If your network doesn't require a proxy, choose "Direct." If your network requires a proxy choose "Manual." Enter the server name and port in the format *http://hostname:port/*.



Once you set a proxy server, you can simply enable/disable it by selecting the Manual or Direct options. There is no need to re-enter the proxy server every time you connect to a network requiring that proxy server. The tool will remember your last server name.

Note that "Direct" will also work if you have the *http_proxy* and *https_proxy* environment variables set. If you prefer to use the environment variables, just choose "Direct" and create/delete the variables depending on whether or not your site needs the proxy.

The settings made in either the Project Creator or Library Manager apply to one another.

17.2 Offloading manifest files

In some locations, `git clone` operations from GitHub may be allowed while raw file access may be restricted. This prevents manifest files from being loaded. In this scenario, the manifest files can be offloaded either to an alternate server or a local location on disk. For details on how to do this, see the following knowledge base article:

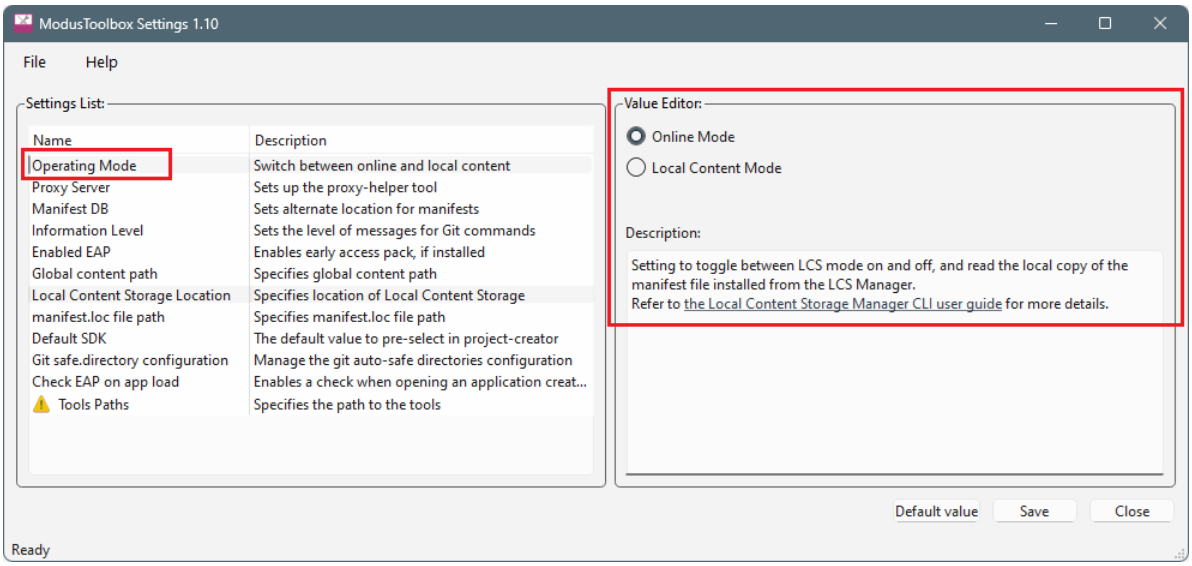
https://community.infineon.com/t5/Knowledge-Base-Articles/Offloading-the-Manifest-Files-of-ModusToolbox-KBA230953/ta-p/252973

17.3 Local content

In the case where network access is impossible, Infineon provides a method to use content (i.e. assets such as libraries, BSPs and code examples) from a local disk. The first step is to download the content that you will require. You must have internet access for this step, but once everything you need is downloaded, you won't need internet access. Downloading is done using the Local Content Storage manager tool which is provided as part of the ModusToolbox™ installer. This is a command line tool that allows you to download and manage assets. You can download everything or just download the specific assets that you need. You can also use the tool to check for and download updates.

Once you have downloaded the assets that you require, switch the tools to use the local content instead of the online content. This can be done in one of two ways:

- In one of the tools that you want to use with local content (Project Creator, BSP Assistant or Library Manager), use the menu option under **Settings** to select **ModusToolbox Settings**. Then choose **Operating Mode**.



- Select **Online Mode** to operate normally online; use **Local Content** mode to use local content offline.

Details on using the Local Content Storage manager command line tool can be found in the ModusToolbox™ documentation under *<Install Directory>/ModusToolbox/tools_<version>/lcs-manager-cli/docs/lcs-manager-cli.pdf*.

18 Compiler optimization

The default compiler setting is "Debug." If you want to change this to "Release" to increase the optimization that is done, you must do 2 things:

- Change the value of the variable `CONFIG` in the *Makefile* to "Release."
- Regenerate or update any IDE launch configurations. This is necessary because the build output files go in a directory that has the `CONFIG` setting it the path. Therefore, if you don't update the configurations, the program and debug step will use the old build output files.

Note that the definition of Debug and Release are compiler dependent. For GCC_ARM, you can find the settings in *recipe-make-cat1a/<version>/make/toolchains/GCC_ARM.mk*:

```
ifeq ($(CONFIG),Debug)
CY_TOOLCHAIN_DEBUG_FLAG=-DDEBUG
CY_TOOLCHAIN_OPTIMIZATION=-Og
else ifeq ($(CONFIG),Release)
CY_TOOLCHAIN_DEBUG_FLAG=-DNDEBUG
CY_TOOLCHAIN_OPTIMIZATION=-Os
else
CY_TOOLCHAIN_DEBUG_FLAG=
CY_TOOLCHAIN_OPTIMIZATION=
endif
```

You can also set the value of `CONFIG` to a custom name (e.g. "Custom") and then use the CFLAGS variable in the *Makefile* to set the optimization parameters as you wish.

19 Exercises (part 2)

For the following exercises, the CY8CKIT-062S2-43012KIT_PSC3M5_EVK kit is shown, but you can choose to use any available BSP to explore the family or solution of your choice. The exact steps and starting application that you use may be different than what is shown in the following exercises, but the concepts will be the same.

Exercise 3: Use the Device Configurator to change BSP settings

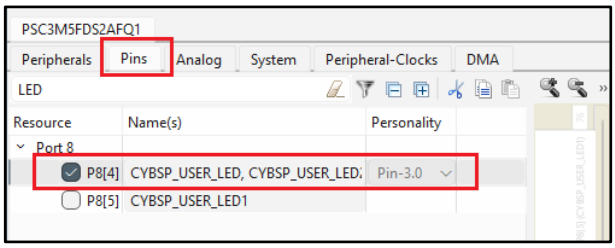
In this exercise, you will use the Device Configurator. The instructions demonstrate changing a pin name to swap LED pin assignments, and making the LED blink faster. However, you can make any change you want.

1. Use the existing application, or create a new one from the Hello World example and call it **ch02_ex04_custom**.

2. Run the Device Configurator from within the application:

   - CLI: Enter the command `make device-configurator`.

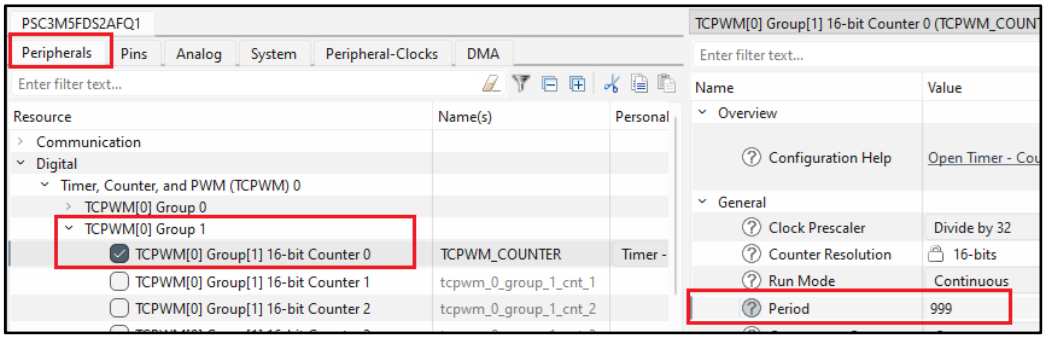
3. Once the Device Configurator opens, make sure you are on the MCU tab, select the **Pins** tab and enter LED in the filter text box. Move the definition for `CYBSP_USER_LED` from `P8[5]` to `P8[4]`. Since the code in *main.c* uses the name `CYBSP_USER_LED`, this change will cause a different LED to blink.



Note: *Don't forget the comma and do NOT add in any spaces as they will be converted to underscores.*

Note: *Make sure to configure the selected pin to match previous pin.*

4. Click on the **Peripherals** tab, and click on the TCPWM peripheral that is already enabled (it has an enabled check box). Change the **Period** parameter to a smaller value (e.g., 999) so that the LED will blink faster this time.



5. Save the change and exit the Device Configurator.

6. Build and program. Observe the blinking LED is now the other user LED, appearing orange instead of blue, and it blinking at a faster rate.

Note: *The solution project uses the templates mechanism described in section 5.3 to provide the updated Device Configurator file. During application creation, that file is copied into the BSP.*

Exercise 4: Use the BSP Assistant

In this exercise, you will create a custom BSP with the BSP Assistant and then use it in an application.

1. Start the BSP Assistant.
2. Create a new BSP called *MyBSP*. You can save it to any convenient location on disk.

Note: *Select the kit that you used for the earlier exercises as the BSP Source Template so that the starting BSP will work correctly on your kit. For example, if you used the KIT-PSC3M5-EVK, select that kit as the Source Template.*

3. Open the Device Configurator from the BSP Assistant.
4. Once the Device Configurator opens, make sure you are on the MCU tab (PSC3M5FDS2AFQ1), select the **Pins** tab and enter LED in the filter text box. Move the definition for `CYBSP_USER_LED` from `P8[5]` to `P8[4]`.
5. Exit the Device Configurator.
6. Explore other settings in the BSP such as dependencies and capabilities.
7. Exit the BSP Assistant.
8. Create a new application. Instead of selecting the KIT-PSC3M5-EVK, use the **Browse for BSP** button to select your custom BSP.
9. Build and program the application. Observe the blinking LED is now the blue LED from the RGB LED.

Note: *The solution has two directories for this exercise. The directory key_ch02_ex05_bsp contains a custom BSP created from the KIT-PSC3M5-EVK (known as MyBSP). The directory key_ch02_ex05_application contains the archive that uses the custom BSP. If you want to use the solution project, you will need to select the **Browse…** functionality inside Project Creator to specify both the BSP and the application.*

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition: 2026-06-25

Published by
Infineon Technologies AG
Am Campeon 1-15
85579 Neubiberg, Germany

© 2026 Infineon Technologies AG.
All rights reserved

Do you have a question about this document?
Email: erratum@infineon.com

Document reference:

Important Notice
Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.
For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.
Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.
If this document is marked as 'Preliminary', 'Target', 'Draft' or 'Proposal', Infineon reserves the right to change all information given in this document at any time without notice. Subject to the development and release of a Product for series supply by Infineon, the technical specifications of the Product are set forth in the relevant datasheet provided by Infineon without such marking.
The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.
Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.
Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).
If the Product includes security features:
Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.
If this document includes or references software:
The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved.
Therefore, you may use the software only as provided in the software license agreement accompanying the software.
If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited.
For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com.